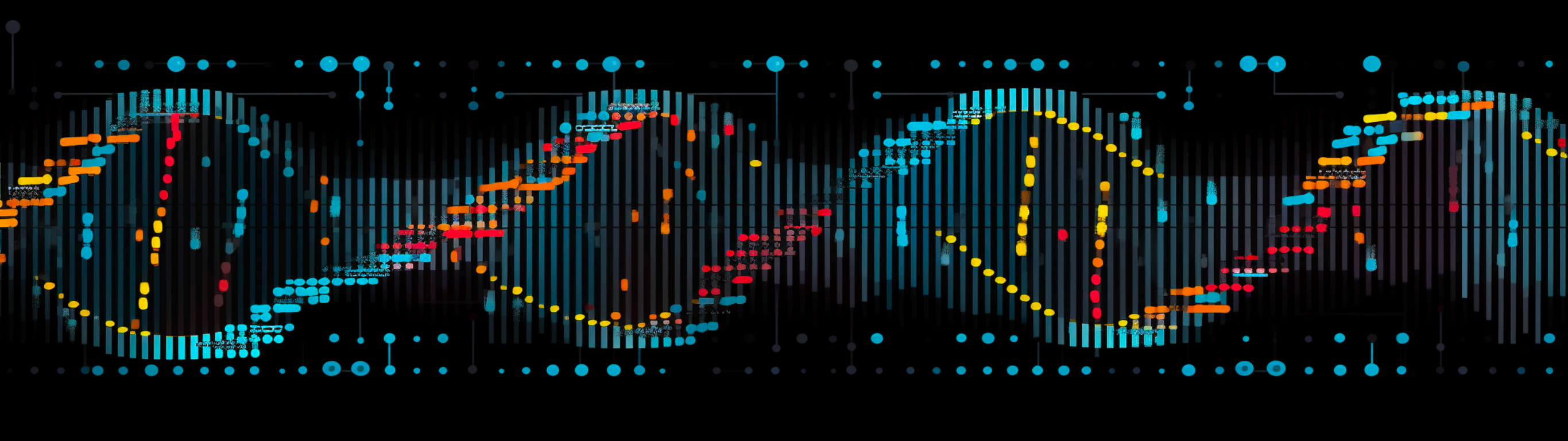




Introduction to bioinformatics

Chapter 2 Alignment and NGS Reads Mapping - BLAST

Jiaxing Chen

Outline

- Sequence Alignment
 - Global Alignment by Dynamic Programming
 - Alignment - From Global to Local
- NGS Reads Mapping - BLAST

Sequence Alignment

Biological Question:

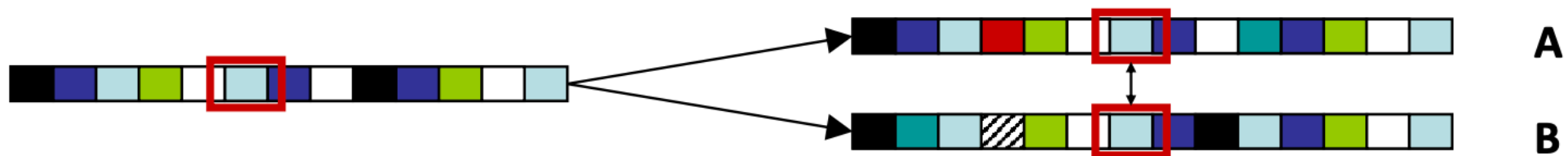
“How can we determine the similarity between two sequences?”

Why is it important?

- Similar sequence → Similar structure → Similar function (The “*Sequence-to-Structure-to-Function Paradigm*”)
- Similar sequence → Common ancestor (“*Homology*”)

Sequence Alignment in Biology

The purpose of a sequence alignment is to line up all residues in the inputted sequence(s) for **maximal level of similarity**, in the sense of **their functional or evolutionary relationship**.



Pairwise Sequence x

www.ebi.ac.uk/Tools/psa/

EMBL-EBI

ServicesResearchTrainingIndustryAbout us

Pairwise Sequence Alignment

ShareFeedback

Tools > Pairwise Sequence Alignment

Pairwise Sequence Alignment is used to identify regions of similarity that may indicate functional, structural and/or evolutionary relationships between two biological sequences (protein or nucleic acid).

By contrast, **Multiple Sequence Alignment (MSA)** is the alignment of three or more biological sequences of similar length. From the output of MSA applications, homology can be inferred and the evolutionary relationship between the sequences studied.

Global Alignment

Global alignment tools create an end-to-end alignment of the sequences to be aligned. There are separate forms for protein or nucleotide sequences.

Needle (EMBOSS) ?

EMBOSS Needle creates an optimal global alignment of two sequences using the Needleman-Wunsch algorithm.

[Protein](#) [Nucleotide](#)

Stretcher (EMBOSS) ?

EMBOSS Stretcher uses a modification of the Needleman-Wunsch algorithm that allows larger sequences to be globally aligned.

[Protein](#) [Nucleotide](#)

Local Alignment

Local alignment tools find one, or more, alignments describing the most similar region(s) within the sequences to be aligned. There are separate forms for protein or nucleotide sequences.

Water (EMBOSS) ?

EMBOSS Water uses the Smith-Waterman algorithm (modified for speed enhancements) to calculate the local alignment of two sequences.

[Protein](#) [Nucleotide](#)

Matcher (EMBOSS) ?

EMBOSS Matcher identifies local similarities between two sequences using a rigorous algorithm based on the LALIGN application.

[Protein](#) [Nucleotide](#)

LALIGN ?

LALIGN finds internal duplications by calculating non-intersecting local alignments of protein or DNA sequences.

Genomic Alignment

Genomic alignment tools concentrate on DNA (or to DNA) alignments while accounting for characteristics present in genomic data.

Wise2DBA ?

Wise2DBA (DNA Block Aligner) aligns two sequences under the assumption that the sequences share a number of colinear blocks of conservation separated by potentially large and varied lengths of DNA in the two sequences.

[Launch Wise2DBA](#)

GeneWise ?

GeneWise compares a protein sequence to a genomic DNA sequence, allowing for introns and frameshifting errors.

[Launch GeneWise](#)

PromoterWise ?

PromoterWise compares two DNA sequences allowing for inversions and translocations, ideal for promoters.

[Launch PromoterWise](#)


```

=====
#
# Aligned_sequences: 2
# 1: HBA_HUMAN
# 2: HBB_HUMAN
# Matrix: EBLOSUM62
# Gap_penalty: 10.0
# Extend_penalty: 0.5
#
# Length: 149
# Identity:      65/149 (43.6%)
# Similarity:    90/149 (60.4%)
# Gaps:          9/149 ( 6.0%)
# Score: 292.5
#
=====

```

```

HBA_HUMAN      1 MV-LSPADKTNVKAANGKVGAGHAGEYGAELERMFLSFPTTKTYFPHF-D 48
|| |:|:|:|.|.||| | :..|.|.|||.|:|:|.|-|:|:|.|| |
HBB_HUMAN      1 MVHLTPEEKSAVTALWGKV--NVDEVGGEALGRLLVVYPWTQRFFESFGD 48

HBA_HUMAN     49 LS-----HGSAQVKGHGKKVADALTNVAHVDDMPNALSALSDLHAHKLR 93
||      .|:|:|:|.||| |.|.|:|:|.||:|:|:|.||:|:|:|.||.
HBB_HUMAN     49 LSTPDVAVMGNPVKVKAHGKKVLGAFSDGLAHLNLKGTFTATLSELHCDKLH 98

HBA_HUMAN     94 VDPVNFKLLSHCLLVTLAAHLPAEFTPAVHASLDKFLASVSTVLTSKYR 142
|||.||:|:|:|.||:|.|||.|||.|||.|||.|||.|||.|||.|||.
HBB_HUMAN     99 VDPENFRLLGNVLVCVLAHHFGKEFTPPVQAAYQKVVAGVANALAHKYH 147

```

	C	S	T	P	A	G	N	D	E	Q	H	R	K	M	I	L	V	F	Y	W	
C	9																				C
S	-1	4																			S
T	-1	1	5																		T
P	-3	-1	-1	7																	P
A	0	1	0	-1	4																A
G	-3	0	-2	-2	0	6															G
N	-3	1	0	-2	-2	0	6														N
D	-3	0	-1	-1	-2	-1	1	6													D
E	-4	0	-1	-1	-1	-2	0	2	5												E
Q	-3	0	-1	-1	-1	-2	0	0	2	5											Q
H	-3	-1	-2	-2	-2	-2	1	-1	0	0	8										H
R	-3	-1	-1	-2	-1	-2	0	-2	0	1	0	5									R
K	-3	0	-1	-1	-1	-2	0	-1	1	1	-1	2	5								K
M	-1	-1	-1	-2	-1	-3	-2	-3	-2	0	-2	-1	-1	5							M
I	-1	-2	-1	-3	-1	-4	-3	-3	-3	-3	-3	-3	-3	1	4						I
L	-1	-2	-1	-3	-1	-4	-3	-4	-3	-2	-3	-2	-2	2	2	4					L
V	-1	-2	0	-2	0	-3	-3	-3	-2	-2	-3	-3	-2	1	3	1	4				V
F	-2	-2	-2	-4	-2	-3	-3	-3	-3	-3	-1	-3	-3	0	0	0	-1	6			F
Y	-2	-2	-2	-3	-2	-3	-2	-3	-2	-1	2	-2	-2	-1	-1	-1	-1	3	7		Y
W	-2	-3	-2	-4	-3	-2	-4	-4	-3	-2	-2	-3	-3	-1	-3	-2	-3	1	2	11	W
	C	S	T	P	A	G	N	D	E	Q	H	R	K	M	I	L	V	F	Y	W	

Affine gap penalty: **opening** a gap receives a penalty of **d**; **extending** a gap receives a penalty of **e**. So the total Penalty for a gap with length n would be:

$$\text{Penalty} = d + (n-1) * e$$

Final Score = (sum of substitution scores) + (-1) * (sum of Gap Penalty)

Global Alignment by Dynamic Programming

Pairwise Sequence Alignment: in Maths

- Input data:
 - Two sequences $S1$ and $S2$
- Parameter(s)
 - A **scoring function** f for
 - Substitutions
 - Gaps
- Output:
 - The **optimal alignment** of $S1$ and $S2$, which has the **maximal score**.

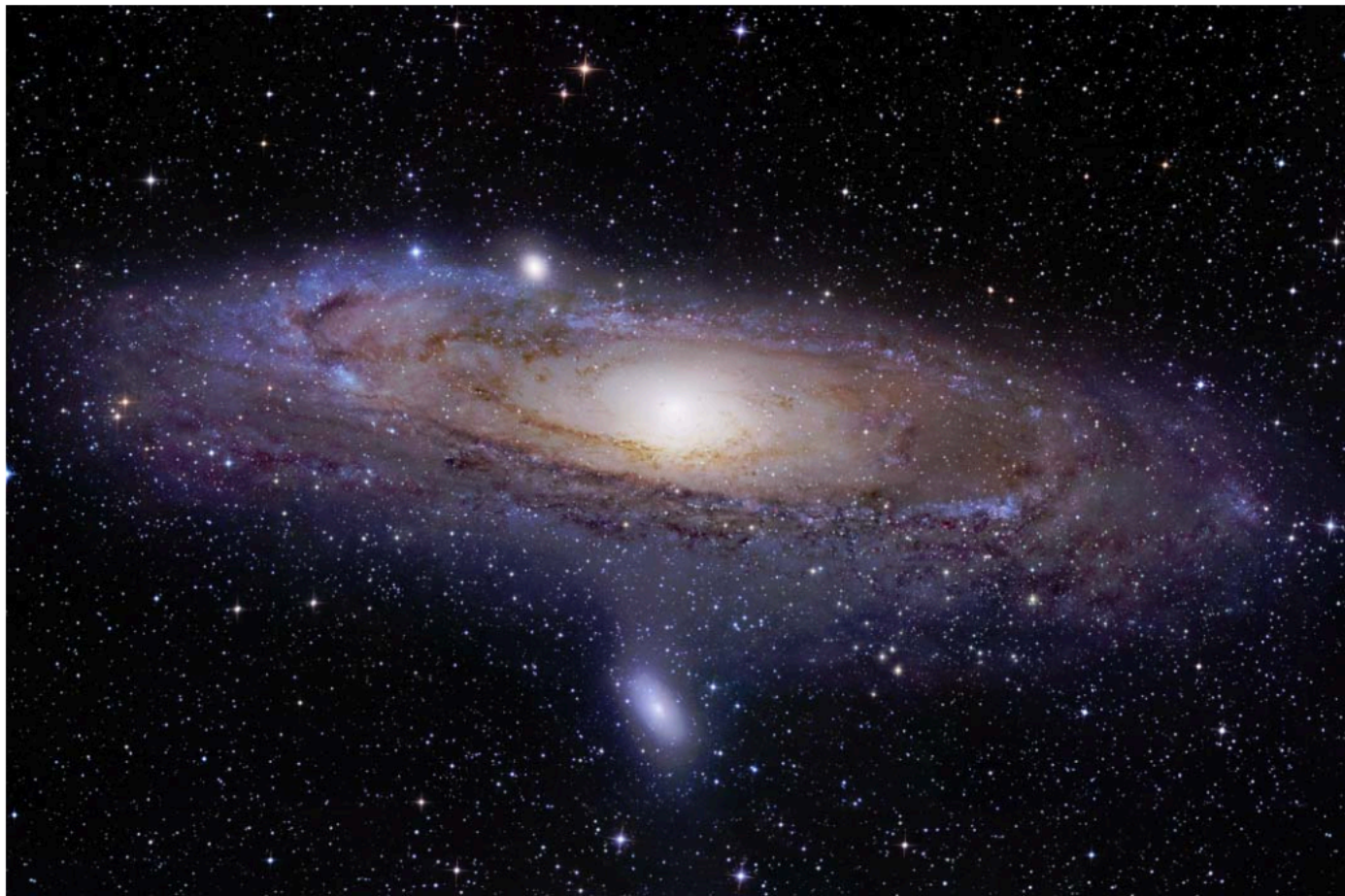
$$\arg \max_{ali} (f(ali(S1, S2)))$$

Sequence Alignment: Enumerate?

LSPADK	L-SPADK	L-SPADK
LTPEEK	LTPEEK-	LT-PEEK
-----LSPADK	L-S-P-A-D-K-	
LTPEEK-----	-L-T-P-E-E-K	

$$\binom{2n}{n} = \frac{(2n)!}{(n!)^2}$$

$$\binom{2n}{n} = \frac{(2n)!}{(n!)^2} = \frac{(2 * 300)!}{(300!)^2} \approx 7 \times 10^{88}$$



The visible universe is estimated to contain $10^{78} \sim 10^{80}$ atoms (Source: wikianswers) only !

Sequence Alignment:
What is the computational Algorithm?

MVHLT P

HBA_HUMAN	1 MV-LSPADKTNVKAAWGKVGAHAGEYGAELERMFLSFPTTKTYFPHF-D	48
	: .: .:. . . :.. . . .:... .:.:. ..	
HBB_HUMAN	1 MVHLTPEEKSAVTALWGKV--NVDEVGGEALGRLLVVYPWTQRFFESFGD	48
HBA_HUMAN	49 LS-----HGSAQVKGHGKKVADALTNAVAHVDDMPNALSALSDLHAHKLR	93
	. .: .:.:.:. . .: :..... .:. .. .	
HBB_HUMAN	49 LSTPDAVMGNPKVKAHGKKVLGAFSDGLAHL DNLKGT FATLSELHC DK LH	98
HBA_HUMAN	94 VDPVNFKLLSHCLLVTLAAHLPAEFTPAVHASLDKFLASVSTVLTSKYR	142
	. : .:. .:.:. .:. .:. .. .	
HBB_HUMAN	99 VDPENFRLLGNVLVCVLAHHFGKEFTP PVQAAYQKVVAGVANALAHKYH	147

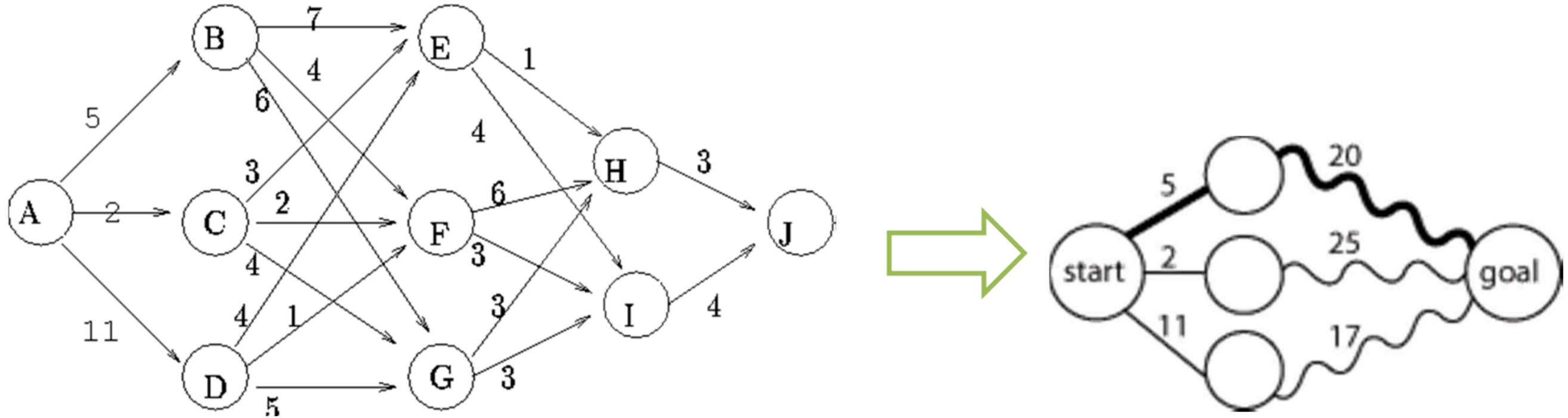
A residue can either

- Align to other residue, or
- Align to a gap

S	S	-
T	-	T

Dynamic Programming

Dynamic Programming solves problems by combining the solutions to **sub-problems**.



Sequence alignment with Dynamic Programming: the Formula

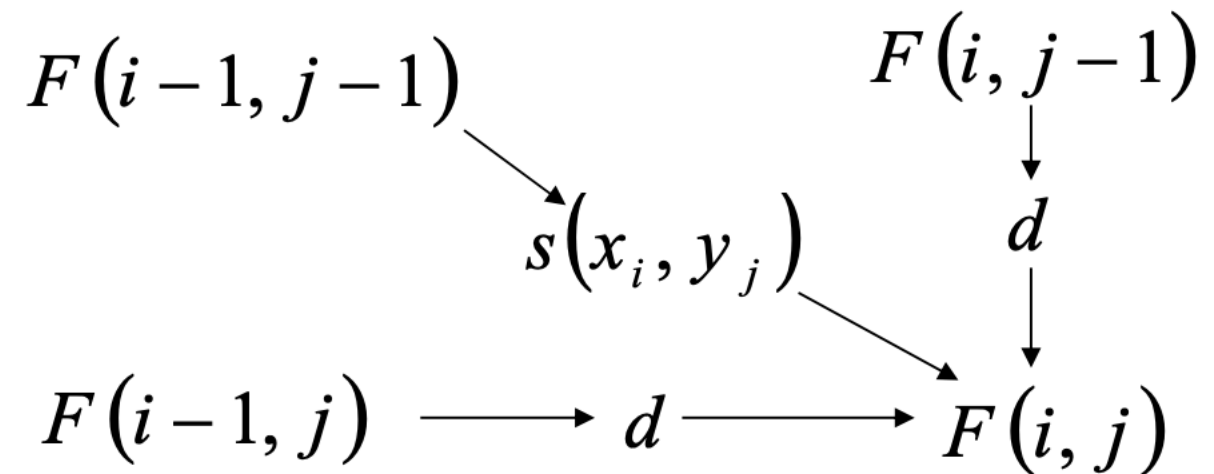
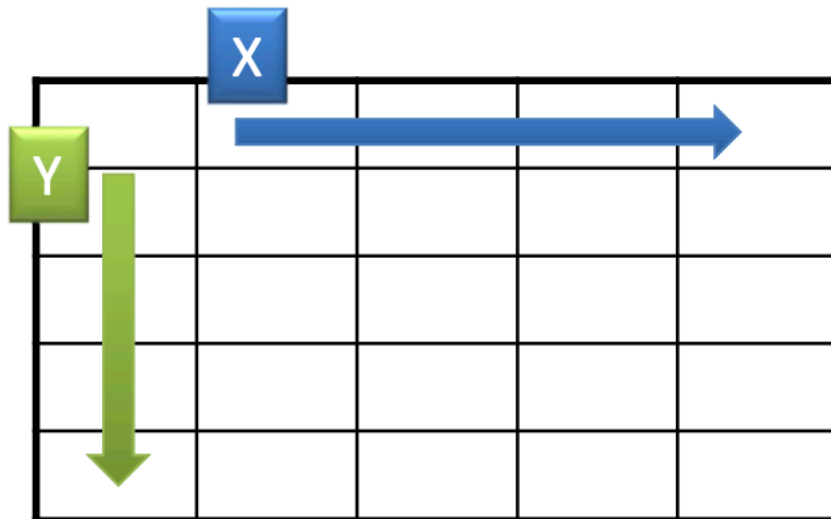
- Align two sequences: x and y
 - $F(i,j)$ is the score of the best alignment between $x_{1\dots i}$ and $y_{1\dots j}$
 - $s(A,B)$ is the score for substituting A with B ; d is the (linear) gap penalty

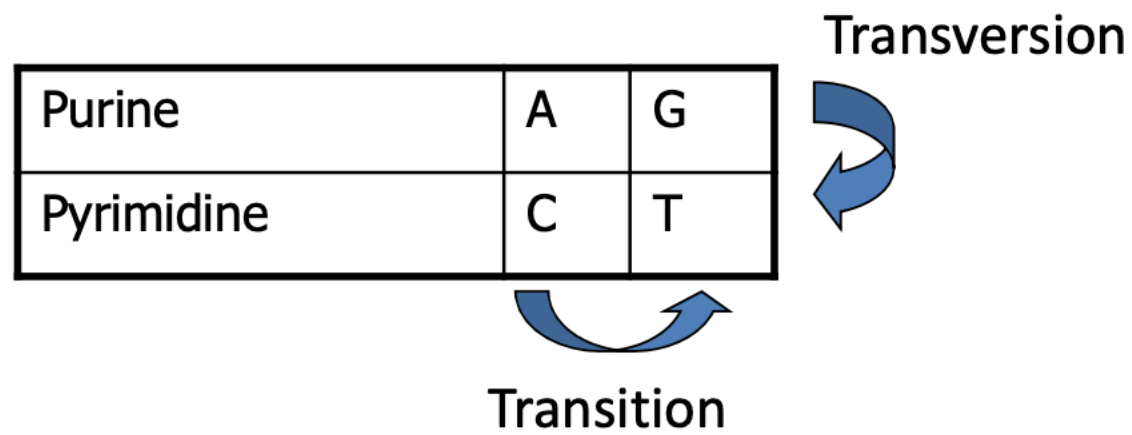
$$F(0,0) = 0$$

$$F(i,j) = \max \begin{cases} F(i-1, j-1) + s(x_i, y_j) & \mathbf{x_i \text{ aligned to } y_j} \\ F(i-1, j) + d & \mathbf{x_i \text{ aligned to a gap}} \\ F(i, j-1) + d & \mathbf{y_j \text{ aligned to a gap}} \end{cases}$$

$$F(0,0) = 0$$

$$F(i, j) = \max \begin{cases} F(i-1, j-1) + s(x_i, y_j) & \mathbf{x_i \text{ aligned to } y_j} \\ F(i-1, j) + d & \mathbf{x_i \text{ aligned to a gap}} \\ F(i, j-1) + d & \mathbf{y_j \text{ aligned to a gap}} \end{cases}$$





Scoring Nucleotide

A nucleotide substitution matrix:

	A	C	G	T
A	2	-7	-5	-7
C	-7	2	-7	-5
G	-5	-7	2	-7
T	-7	-5	-7	2

Input Sequence 1: AAG

Input Sequence 2: AGC

	A	C	G	T
A	2	-7	-5	-7
C	-7	2	-7	-5
G	-5	-7	2	-7
T	-7	-5	-7	2

For simplicity, let's set (i.e. **linear gap penalty**)

gap OPEN (d) = gap EXTEND (e) = -5

GAC-AT

C-ACAT

$(-7) + (-5) + (-7) + (-5) + 2 + 2 = -20$

Dynamic Programming Matrix

		A	A	G
A				
G				
C				

DP for sequence alignment: Example

	A	C	G	T
A	2	-7	-5	-7
C	-7	2	-7	-5
G	-5	-7	2	-7
T	-7	-5	-7	2

Find the optimal alignment of AAG and AGC.

Use a linear gap penalty of $d=-5$.

		A	A	G
	0			
A				
G				
C				

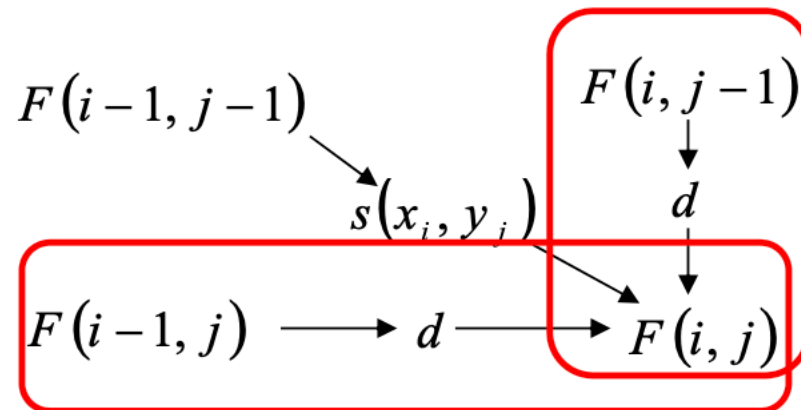
$$F(0,0)=0$$

$$F(i,j)=\max\begin{cases} F(i-1,j-1)+s(x_i,y_j) \\ F(i-1,j)+d \\ F(i,j-1)+d \end{cases}$$

DP for sequence alignment: Example

	A	C	G	T
A	2	-7	-5	-7
C	-7	2	-7	-5
G	-5	-7	2	-7
T	-7	-5	-7	2

Find the optimal alignment of AAG and AGC.
Use a linear gap penalty of $d=-5$.

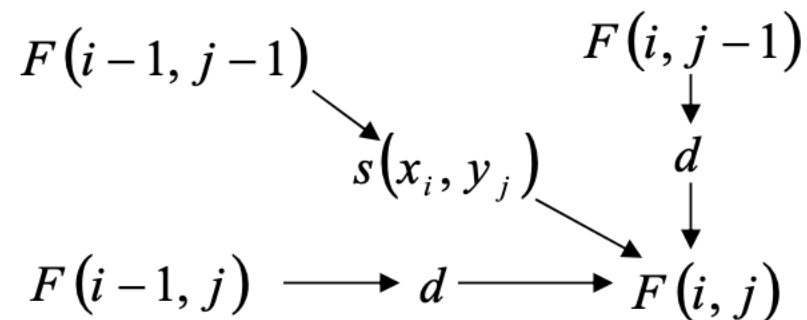


		A	A	G
	0	-5	-10	-15
A	-5			
G	-10			
C	-15			

DP for sequence alignment: Example

	A	C	G	T
A	2	-7	-5	-7
C	-7	2	-7	-5
G	-5	-7	2	-7
T	-7	-5	-7	2

Find the optimal alignment of AAG and AGC.
Use a linear gap penalty of $d=-5$.



		A	A	G
	0	-5	-10	-15
A	-5	2	-3	-8
G	-10	-3	-3	-1
C	-15	-8	-8	-6

Traceback: Decode the “Real” Alignment

- Trace back to the **upper left**. Each arrow introduces **one** symbol at the end of each aligned sequence.

A A G -
- A G C
A A G -
A - G C

		A	A	G
	0	-5		
A		2	-3	
G				-1
C				-6

Alignment-From Global to Local

A A G -
 - A G C
 A A G -
 A - G C

		A	A	G
	0	-5		
A		2	-3	
G				-1
C				-6

End-to-end: Global Alignment

Global Alignment: End-to-end

J. Mol. Biol. (1970) 48, 443–453

Needleman–Wunsch algorithm

A General Method Applicable to the Search for Similarities in the Amino Acid Sequence of Two Proteins

SAUL B. NEEDLEMAN AND CHRISTIAN D. WUNSCH

*Department of Biochemistry, Northwestern University, and
Nuclear Medicine Service, V. A. Research Hospital
Chicago, Ill. 60611, U.S.A.*

(Received 21 July 1969)

A computer adaptable method for finding similarities in the amino acid sequences of two proteins has been developed. From these findings it is possible to determine whether significant homology exists between the proteins. This information is used to trace their possible evolutionary development.

The maximum match is a number dependent upon the similarity of the sequences. One of its definitions is the largest number of amino acids of one protein that can be matched with those of a second protein allowing for all possible interruptions in either of the sequences. While the interruptions give rise to a very large number of comparisons, the method efficiently excludes from consideration those comparisons that cannot contribute to the maximum match.

Comparisons are made from the smallest unit of significance, a pair of amino acids, one from each protein. All possible pairs are represented by a two-dimensional array, and all possible comparisons are represented by pathways through the array. For this maximum match only certain of the possible pathways must be evaluated. A numerical value, one in this case, is assigned to every cell in the array representing like amino acids. The maximum match is the largest number that would result from summing the cell values of every pathway.

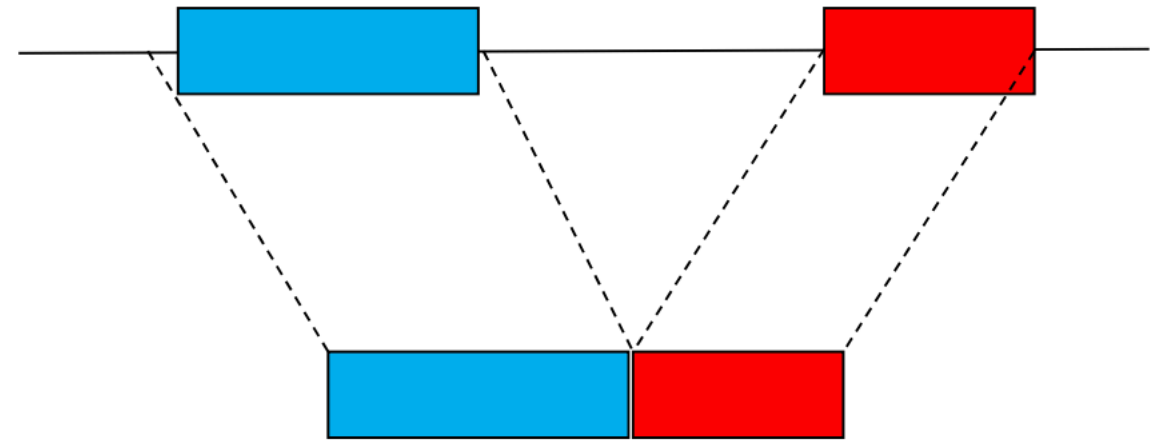
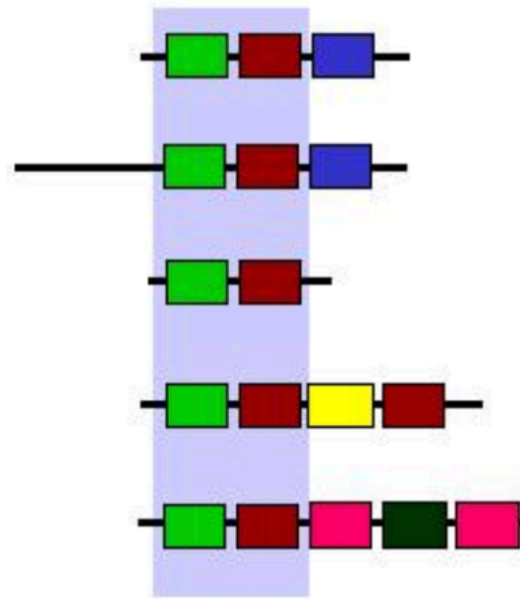
EF-Tu and eEF1-alpha

eRF-3

eIF-2gamma

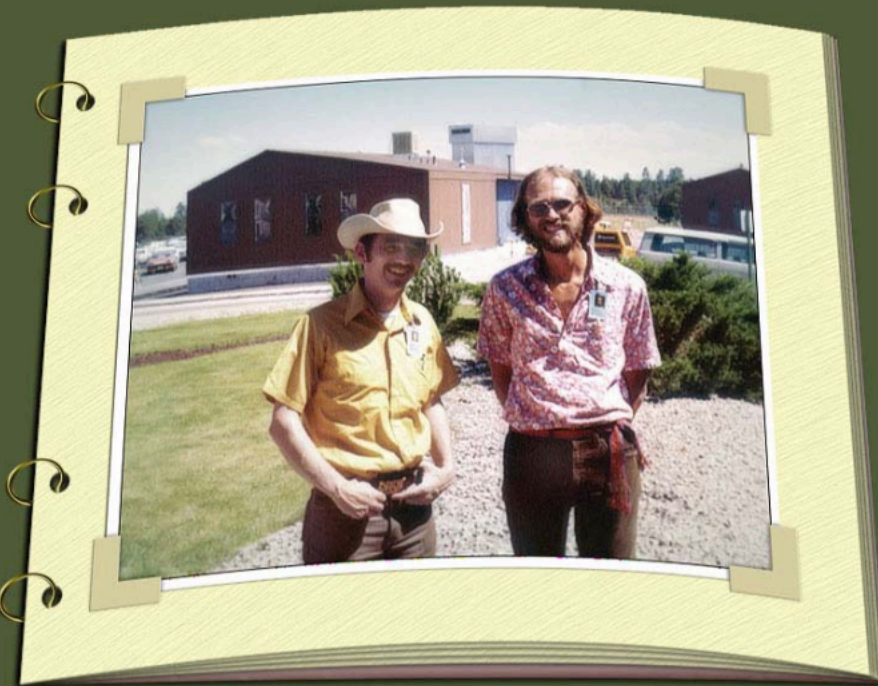
IF-2 and eIF-5b

EF-G and eEF-2



Identify similar **sub-sequence**

Smith and Waterman at Los Alamos, New Mexico
Photo by David Lipman, taken summer of 1980



(<http://www.cmb.usc.edu/people/msw/SmithWaterman.html>)

J. Mol. Biol. (1981), 147, 195–197

Identification of Common Molecular Subsequences

The identification of maximally homologous subsequences among sets of long sequences is an important problem in molecular sequence analysis. The problem is straightforward only if one restricts consideration to contiguous subsequences (segments) containing no internal deletions or insertions. The more general problem has its solution in an extension of sequence metrics (Sellers 1974; Waterman *et al.*, 1976) developed to measure the minimum number of “events” required to convert one sequence into another.

These developments in the modern sequence analysis began with the heuristic homology algorithm of Needleman & Wunsch (1970) which first introduced an iterative matrix method of calculation. Numerous other heuristic algorithms have been suggested including those of Fitch (1966) and Dayhoff (1969). More mathematically rigorous algorithms were suggested by Sankoff (1972), Reichert *et al.* (1973) and Beyer *et al.* (1979), but these were generally not biologically satisfying or interpretable. Success came with Sellers (1974) development of a true metric measure of the distance between sequences. This metric was later generalized by Waterman *et al.* (1976) to include deletions/insertions of arbitrary length. This metric represents the minimum number of “mutational events” required to convert one sequence into another. It is of interest to note that Smith *et al.* (1980) have recently shown that under some conditions the generalized Sellers metric is equivalent to the

$$F(0,0) = 0$$

$$F(i, j) = \max \begin{cases} F(i-1, j-1) + s(x_i, y_j) \\ F(i-1, j) + d \\ F(i, j-1) + d \end{cases} \quad \text{Global alignment}$$

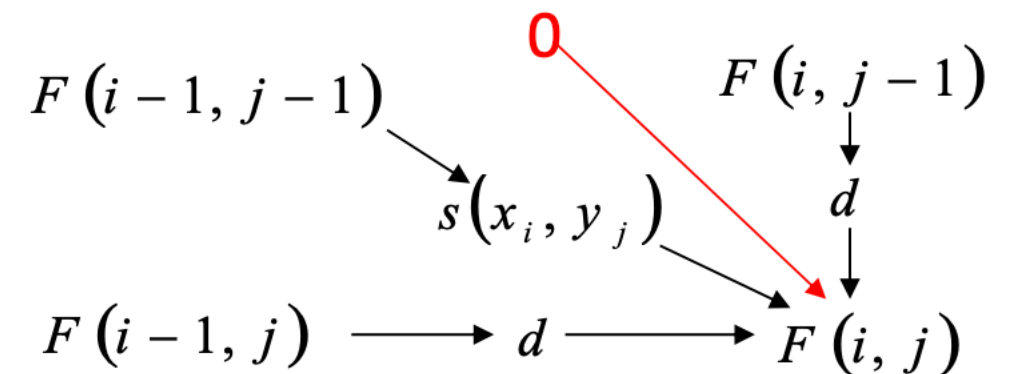
$$F(0,0) = 0$$

$$F(i, j) = \max \begin{cases} F(i-1, j-1) + s(x_i, y_j) \\ F(i-1, j) + d \\ F(i, j-1) + d \\ \boxed{0} \end{cases} \quad \text{Local alignment}$$

DP for Local alignment: Formula

$$F(0, 0) = 0$$

$$F(i, j) = \max \begin{cases} F(i-1, j-1) + s(x_i, y_j) \\ F(i-1, j) + d \\ F(i, j-1) + d \\ 0 \end{cases}$$

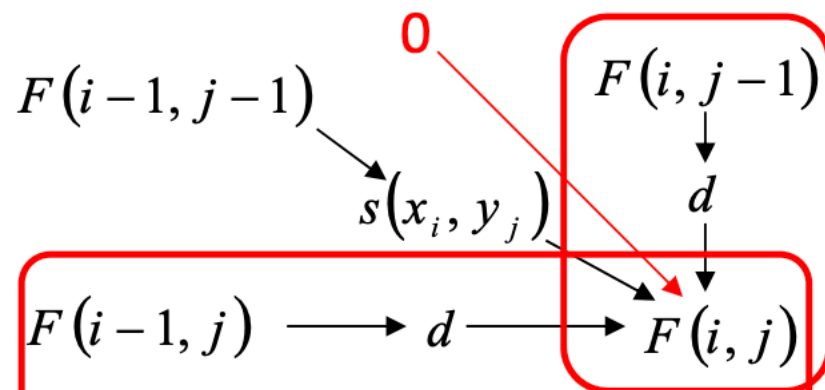


DP for Local alignment: Example

	A	C	G	T
A	2	-7	-5	-7
C	-7	2	-7	-5
G	-5	-7	2	-7
T	-7	-5	-7	2

Find the optimal **local alignment** of AAG and AGC.
Use a linear gap penalty of $d = -5$.

		A	A	G
	0	0	0	0
A	0			
G	0			
C	0			

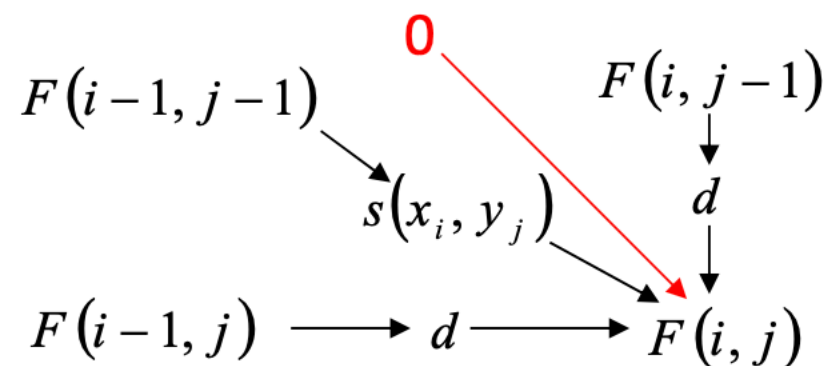


DP for Local alignment: Example

	A	C	G	T
A	2	-7	-5	-7
C	-7	2	-7	-5
G	-5	-7	2	-7
T	-7	-5	-7	2

Find the optimal **local alignment** of AAG and AGC.
Use a linear gap penalty of $d = -5$.

		A	A	G
	0	0	0	0
A	0	2	2	0
G	0	0	0	4
C	0	0	0	0



Traceback: Decode the Local Alignment

- Trace back begins at **the highest score** in the matrix and continues **until you reach 0**.

A G
A G

		A	A	G
	0	0	0	0
A	0	2	2	0
G	0	0	0	4
C	0	0	0	0

Traceback: Decode the Local Alignment

- And also the **secondary best** alignment

A
A

		A	A	G
	0	0	0	0
A	0	2	2	0
G	0	0	0	4
C	0	0	0	0

Global vs. Local

$$F(0,0) = 0$$

$$F(i,j) = \max \begin{cases} F(i-1,j-1) + s(x_i, y_j) \\ F(i-1,j) + d \\ F(i,j-1) + d \end{cases}$$

A	A	G	-
-	A	G	C

A	A	G	-
A	-	G	C

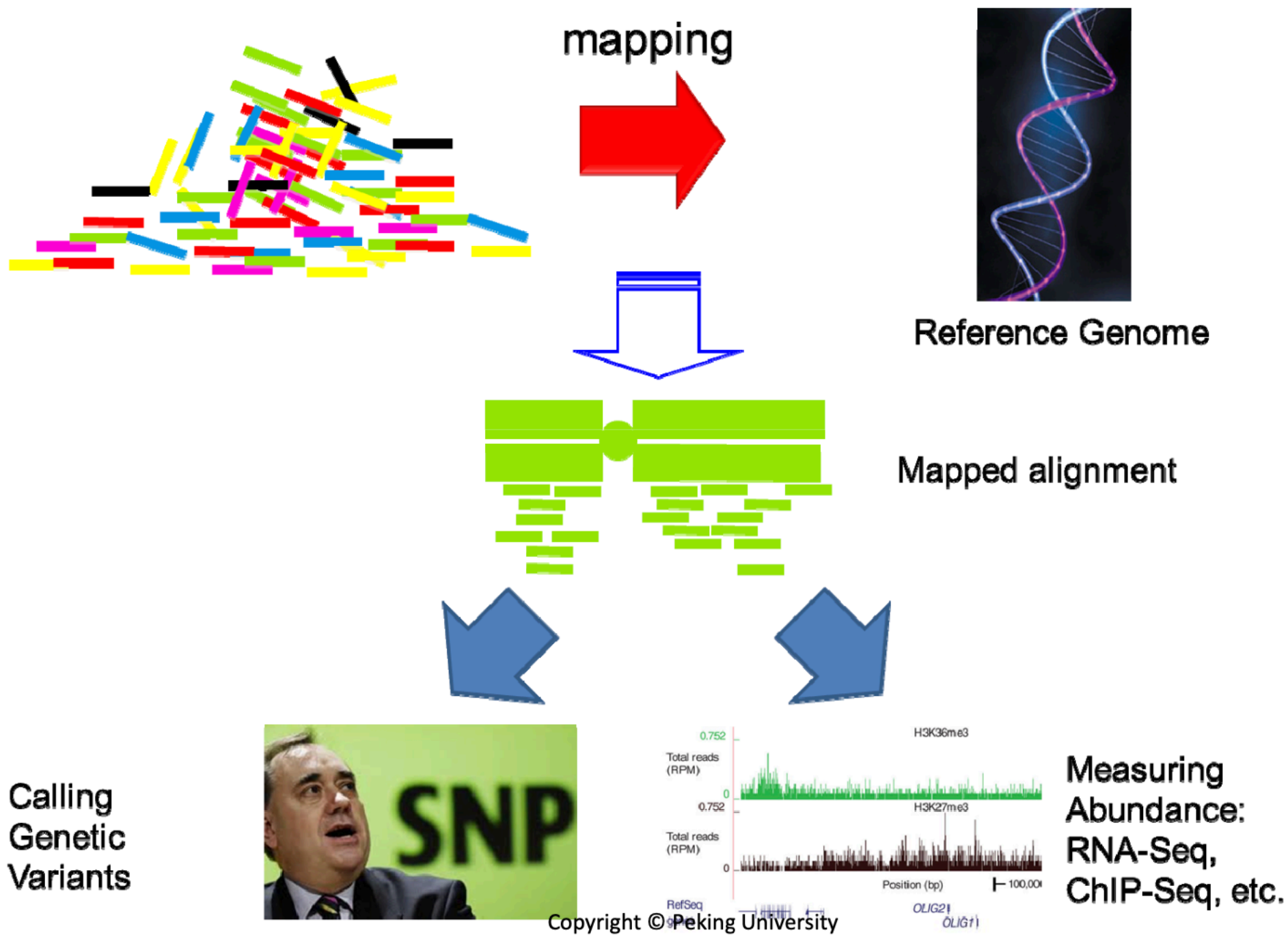
$$F(0,0) = 0$$

$$F(i,j) = \max \begin{cases} F(i-1,j-1) + s(x_i, y_j) \\ F(i-1,j) + d \\ F(i,j-1) + d \\ 0 \end{cases}$$

A	G
A	G

A
A

NGS: Reads Mapping



Mapping: Input Data

- Reference Genome

- Nucleotide

- **Length**: Hundreds of Mb *per* chromosome.

- ~3 Gb in total (for human genome)



mapping



Reference Genome

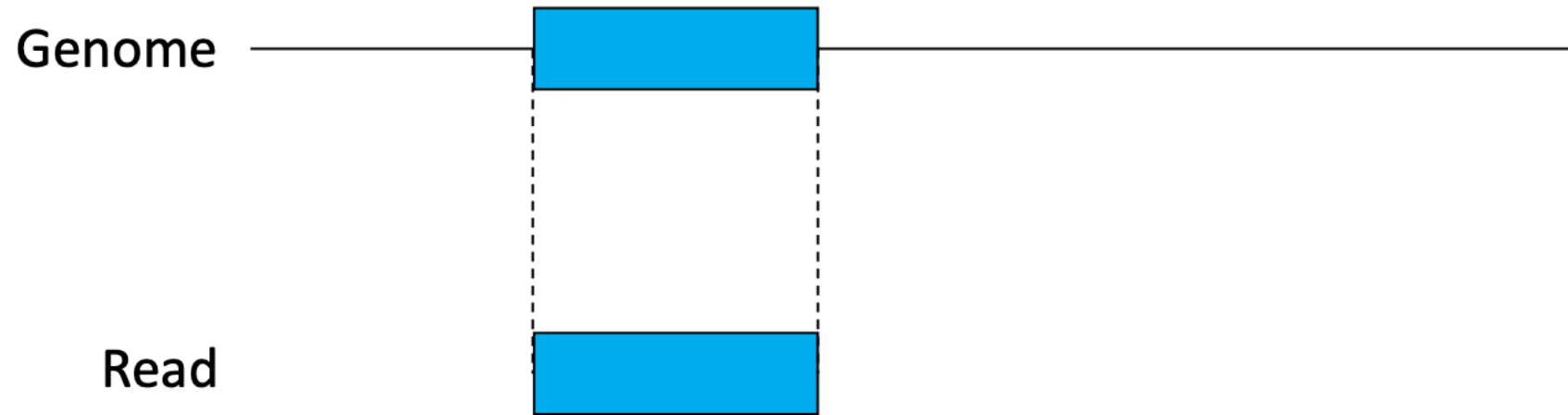
- Reads

- Nucleotide, with **various qualities** (relatively **high error rate**: $1e-2 \sim 1e-5$)

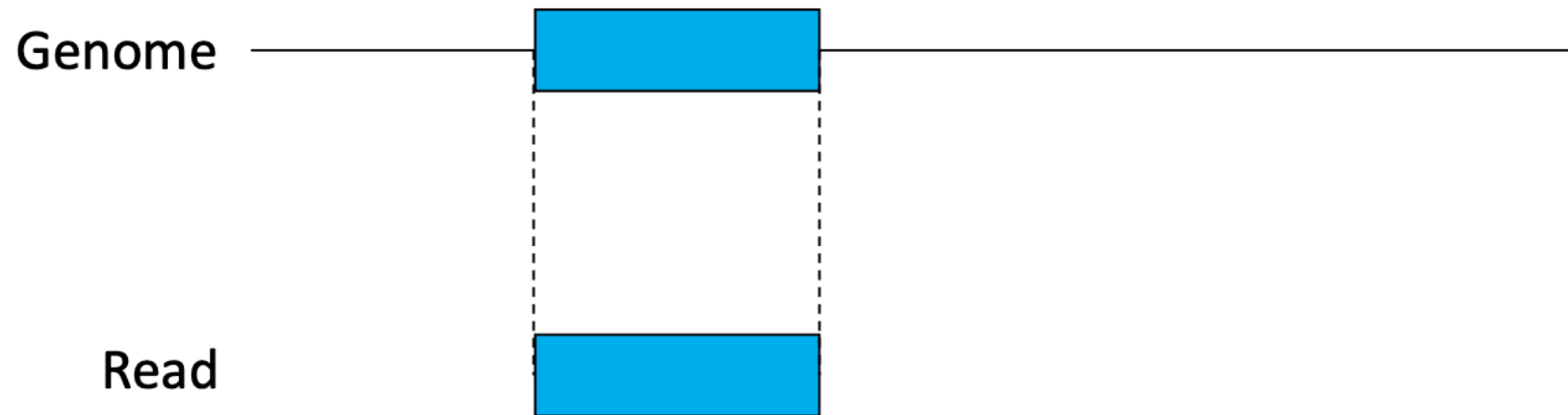
- **Length**: 36~80 bp *per* read

- Hundreds of Gbs *per* run

“Embedded” Alignment

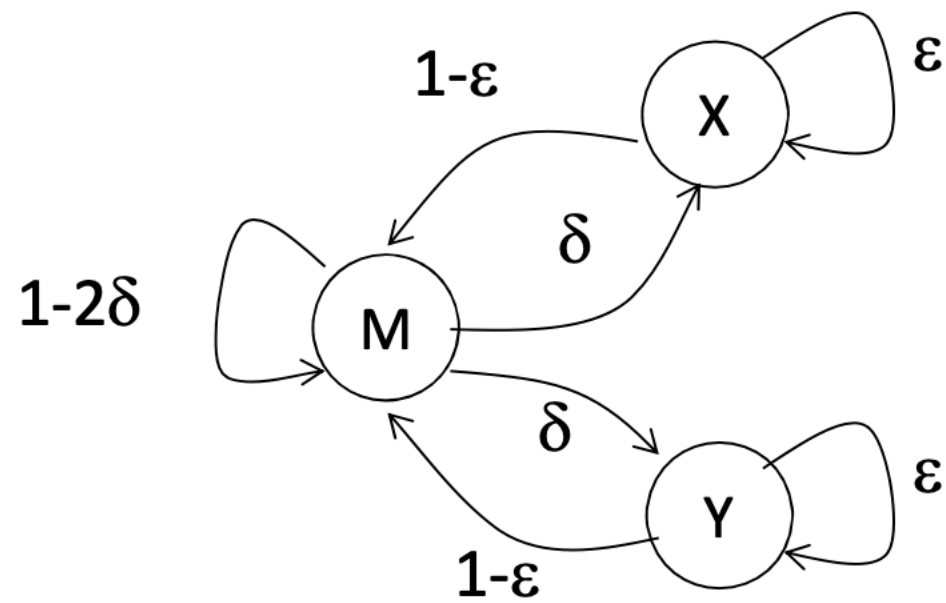


One sequence is “*embedded*” in the other sequence (NGS Reads, PCR primer, *etc.*)



What we need here is actually a hybrid “**global-local**” alignment

- ✓ “**Global**” for short sequence (i.e. NGS Read)
- ✓ But “**Local**” for long sequence (i.e. Reference Genome)
- ✓ In particular, the surrounding “overhang” gaps should be not penalized.



M	<i>Match</i>
X	<i>Insert at sequence X (delete at sequence Y)</i>
Y	<i>Insert at sequence Y (delete at sequence X)</i>

δ	Gap open
ϵ	Gap Extension

	M	X	Y
M	1-2 δ	δ	δ
X	1- ϵ	ϵ	0
Y	1- ϵ	0	0

Genomic chromosome: $m = \text{hundreds of Mb}$

Sequencing Read: $n = 36 \sim 80\text{bp}$



Most of paths will just fail eventually!

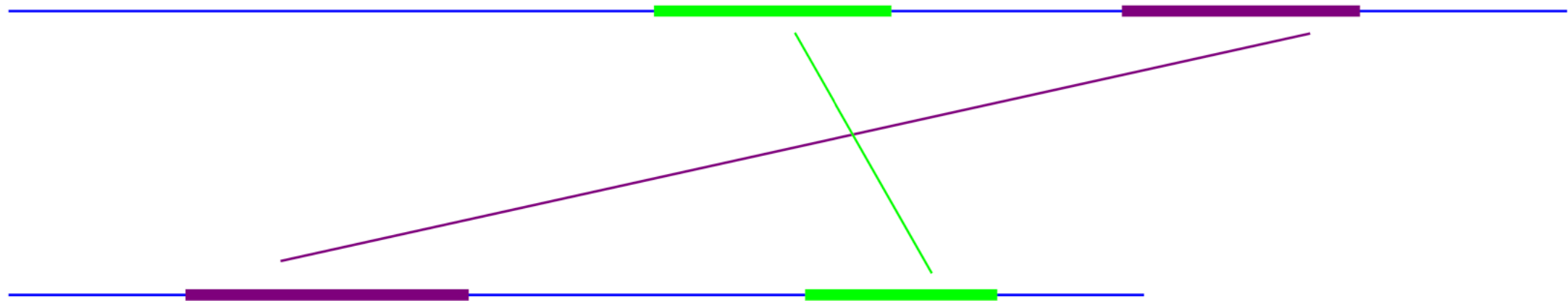


In real world, the speed will be a BIG problem!

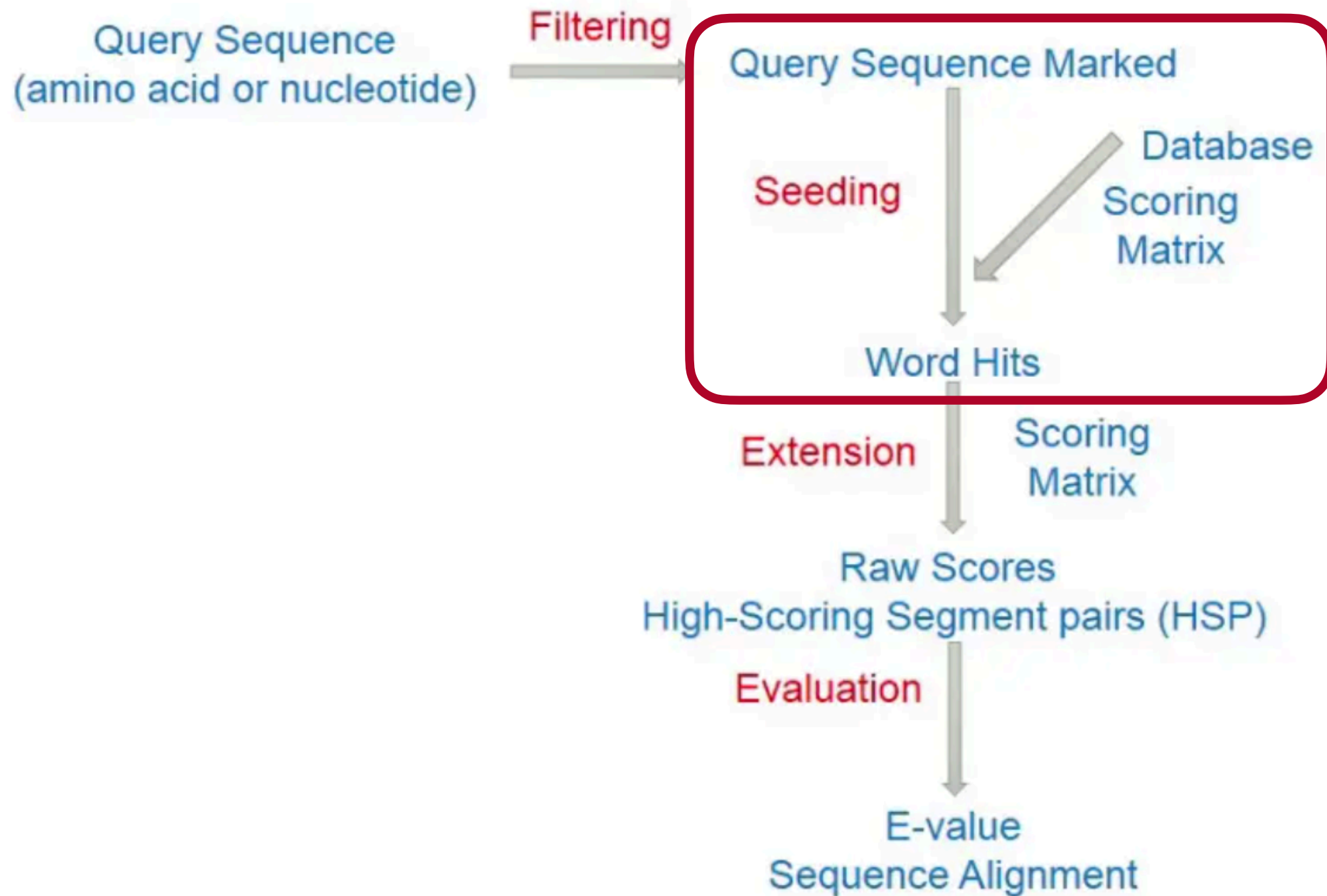
NGS: Reads Mapping - BLAST

BLAST Ideas: Seeding-and-extending

1. Find matches (**seed**) between the query and subject
2. Extend seed into High Scoring Segment Pairs (**HSPs**)
 - Run Smith-Waterman algorithm on the specified region only.
3. Assess the reliability of the alignment.



BLAST



1、seeding

For a given **word length w** (usually 3 for proteins and 11 for nucleotides), slicing the query sequence into multiple continuous “**seed words**”

Query Sequence

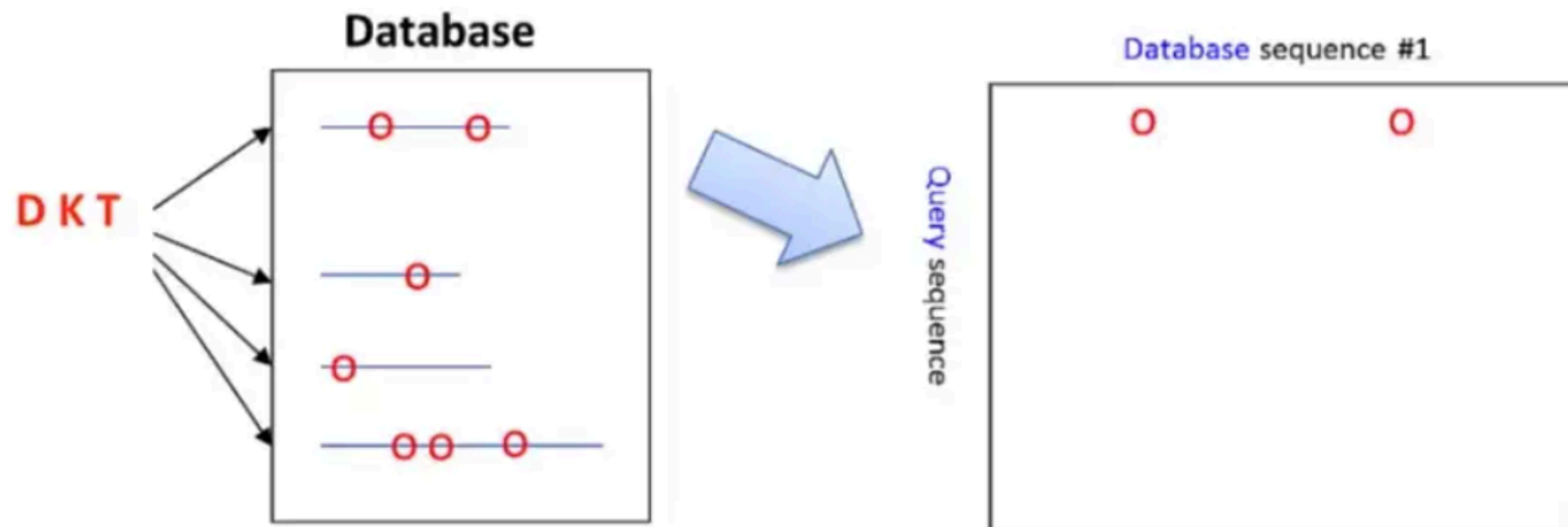
M V L S P A D K T N V K A A W



seeding

2、Index database

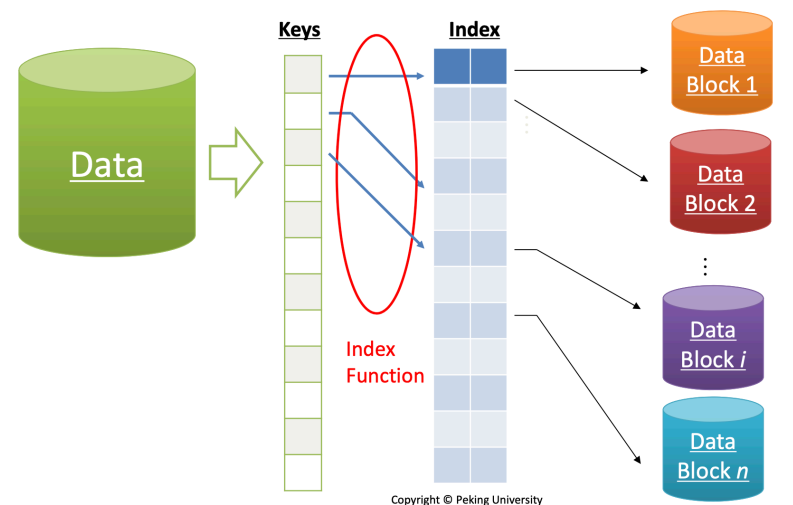
The database was pre-indexed to quickly locate all positions in the database for a given seed.



Seed searching strategy

- Hashing

Allow mismatch



- BWT (Burrows–Wheeler transform)

Suffix Array

BWT string



Alphabetical Index

A

Accelerator, 57
Accelerator Pin, 59
Aperture, 93
Arm, Follower, 60

B

Band, Lock, 180
Barrel, 168 -- 177
Barrel and Mount Base Assembly, 167
Barrel, Mount Base & Receiver, Assembly, 6
Base, 34
Base and Pad Assembly, 23
Base, Mount, 172
Base, Rear Sight, 94
Blank, Rear Sight Screw, 108
Body, 207
Body, Follower Rod, 73
Bolt, 82 -- 83
Bolt Assembly, 79
Bottom Insert, 33
Bracket and Cone Assembly Alternative Const, 200
Bracket and Cone Assembly, M2, 191
Bracket Hinge Mount, 12
Bracket Knob Mount, 13
Bracket Mount Plunger, 15
Bracket, Flash Hider, 192
Bracket, Flash Hider (Alternative Const), 201
Bracket, Mount, 11
Bracket, Mount, Assembly, 10
Buckle, 213
Buckle Slide Loop, 214 -- 216
Bullet Guide, 68
Butt Plate, 131
Butt Plate Assembly, 130
Butt Plate Cap, 127
Butt Plate Cap Pin, 129
Butt Plate Long Screw, 133
Butt Plate Plunger, 132
Butt Plate Plunger Spring, 136
Butt Swivel, 140

C

Cap, Butt Plate, 127
Cartridge Ejector, 84
Cartridge Ejector Assembly, 80
Cartridge Ejector Spring, 88
Catch, Operating Rod, 58
Catch, Operating Rod, Assembly, 56
Cheek Pad Assembly, 22
Clip Ejector, 145
Clip Latch, 70
Clip Latch Pin, 69
Clip Latch Spring, 71
Clips, End, Strap-Type I, 212
Cone, Flash Hider, 190, 199
Cover, Rear Sight, 95
Cylinder, Gas, 113

D

Dog, Indexing, 102
Dog, Indexing, Assembly, 102

E

Early Exterior View M1D Sniper Rifle With M82 Telescope, 218
Early Longitudinal Section M1D Sniper Rifle With M82 Telescope, 219
Early Sectionalized Views M1D Sniper Rifle With M82 Telescope, 220
Ejector, Cartridge, 84
Ejector, Cartridge, Assembly, 80
Ejector, Clip, 145
Extractor, 85
Extractor Spring, 89
Extractor Spring Plunger, 87
Extractor Spring Plunger Assembly, 81
Eyelet - Lacing, 30
Eyelets, Metallic, and Eyelet Washers, Metallic, 24 -- 29

F

Ferrule, Front Hand Guard, 175
Ferrule, Stock, 128
Firing Pin, 86
Flash Hider, M2, 187
Flash Hider, M2 Alternate Construction, 197
Flash Hider, T 37, 114

Flash Hider Bracket (Alternative Const), 201

Flash Hider Cone, 190, 199
Flat-External Tooth Lock Washer, 107
Follower, 61
Follower Arm, 60
Follower Arm Pin, 63
Follower Rod Assembly, 72
Follower Rod Body, 73
Follower Rod L.H. Plate, 75
Follower Rod R.H. Plate, 74
Follower Rod Rivet, 76
Follower Slide, 62
Front Hand Guard, 176
Front Hand Guard Assembly, 174
Front Hand Guard Ferrule 175
Front Hand Guard Spacer, 177
Front Sight, 123
Front Sight Cap Screw, 115

G

Gas Cylinder, 113
Gas Cylinder Lock Screw, 116
Gas Cylinder Lock Screw With Valve Assembly, 120
General Data, Heat Treatment and Finish, 193 -- 196
Geometric Symbols For Dimensioning and Tolerancing, 221
Guard, Hand, Front, 174
Guard, Hand, Front, Assembly, 174
Guard, Hand, Rear, 178
Guard, Trigger, 146
Guard, Trigger, Assembly, 147
Guide, Bullet, 68

H

Hammer, 150 -- 151
Hammer Pin, 155
Hammer Spring, 162
Hammer Spring Housing, 152
Hammer Spring Plunger, 159
Hammer Stop, 149
Handle & Tube Operating Rod Assembly, 51
Handle, Operating Rod, 52
Head, Valve, 117
Heat Treatment and Finish General Data, 193 -- 196
Hider, Flash (T 37), 114
Hider, Flash, M2, 187

Hider, Flash, M2 List of Drawings & Specs, 186

Hinge Pin, 14
Hinge, Mount, Bracket, 12
Hook, 210
Hook Assembly, 208
Housing, Hammer Spring, 152
Housing, Trigger, 153 -- 154

I

Indexing Dog, 109
Indexing Dog Assembly, 102
Insert, Bottom, 33
Insert, Middle, 32
Insert, Top, 31

J

K

Keeper Assembly, 205
Knob Mount Screw, 16
Knob, Mount, Bracket, 13
Knob, Rear Sight Elevating, 106
Knob, Windage, Rear Sight, 97
Knob, Windage, Rear Sight, Assembly, 96

L

Lace, Leather, 36
Lacing Eyelets, 30
Latch, 188, 198
Latch, Clip, 70
Leather, 36
Leather, Concealable, Lace, and Cut Lace, 37 -- 43
Lock Washer, 107
Lock, Rear Sight Nut, 100
Loop, 209
Loops, Slide (For Equipage), 214 -- 216
Lower Band, 180
Lower Band Pin, 179
Lug, Locking, Trigger Guard, 148

M

M1 Gun Sling Alternative Design, 204
M2 Bracket and Cone Assembly, 191
M2 Flash Hider, 187
M2 Flash Hider Bracket, 192

M2 Flash Hider List of Drawings & Specs, 186 (Sheet 1 of 2), 197 (Sheet 2 of 2)

M2 Alternate Construction
M2 Flash Hider Spring Pin, 189
M84 Telescope, 182 -- 183
M84 Telescope Optical System, 184
Middle Insert, 32
Mount Base, 172
Mount Base & Receiver Barrel Assembly, 6
Mount Bracket, 11
Mount Bracket Assembly, 10
Mount Locking Pin, 173
Mount, Telescope, 8

N

Nut, Rear Sight, 99

O

Operating Rod Assembly, National Match (NM), 49
Operating Rod Assembly, 50
Operating Rod Catch, 58
Operating Rod Catch Assembly, 56
Operating Rod Handle, 52
Operating Rod Spring, 55
Operating Rod Tube, 53

P

Pad, 35
Pad, Cheek, Assembly, 22
Parts Identified By Part Number, M1, M1C, M1D Rifle, 222
Pin, Accelerator, 59
Pin, Butt Plate Cap, 129
Pin, Clip Latch, 69
Pin, Firing, 86
Pin, Follower Arm, 63
Pin, Hammer, 155
Pin, Hinge, 14
Pin, Mount, Bracket, 173
Pin, Sear, 156
Pin, Spring, 179
Pin, Spring, M2 Flash Hider, 189
Pin, Trigger, 157
Pin, Trigger (Superseded), 158
Pinion Assembly, Rear Sight Elevating, 101, 110
Piston, Operating Rod, 54

Clayville, OH: 170

Clayville, Ohio: 171

Mt. Pleasant: 153

Presbytery

Maskingum: 236

Secoder Church

Clayville/New Concord, OH: 409

Crawfordsville: 271

Secoders: vii, viii, 141, 148, 149, 150, 153, 154,

156, 162, 167, 169, 170, 171, 172, 176,

184, 185, 186, 189, 190, 195, 196, 197,

198, 199, 201, 204, 210, 211, 212, 210,

231, 236, 237, 238, 239, 240, 241, 242,

243, 253, 256, 257, 258, 260, 261, 264,

265, 266, 268, 297, 323, 411, 416, 488,

506, 659, 799, 804, 806, 812, 814, 836,

845

and Temperance: 179

Iowa: 268

North Carolina: 186

Ohio: 185, 186, 201, 231, 238

Pennsylvania: 238

Scottish: 148, 236

unit action: 627, 628, 630, 632, 634

units: 847

Sharon College: 512, 804

Shelbyville, KY: 399, 436, 570

Shenandoah Valley: 132, 314, 469, 471, 475,

620, 665, 668, 684, 693, 697, 698, 700,

847

Shenandoah Valley Campaign: 671, 684

Shenandoah, Phil: 296, 319, 431, 523, 537, 565,

615, 684, 699, 796, 806, 807, 836

unit action: 400, 428, 430, 431, 432, 526,

542, 559, 620, 622, 645, 646, 665, 666,

667, 671, 683, 684, 693, 697, 698, 699,

700, 765, 769, 770, 789

Sherrin, William Tecumseh: 345, 349, 360,

361, 363, 375, 378, 379, 389, 400, 408,

469, 539, 543, 548, 565, 566, 567, 603,

616, 677, 687, 702, 734, 736, 747, 785,

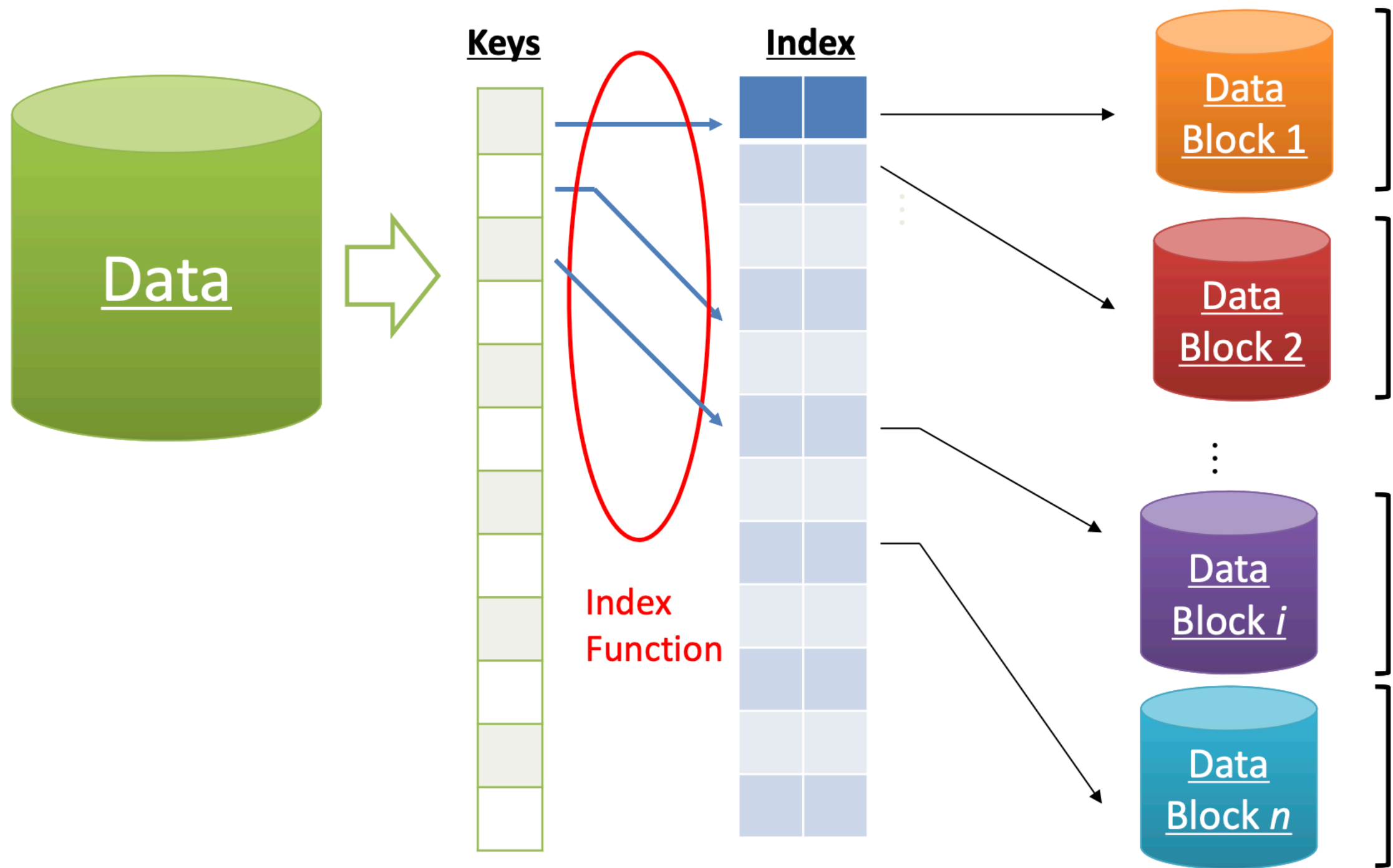
787, 796, 798, 807, 836

family: 615

March to the Sea: 697

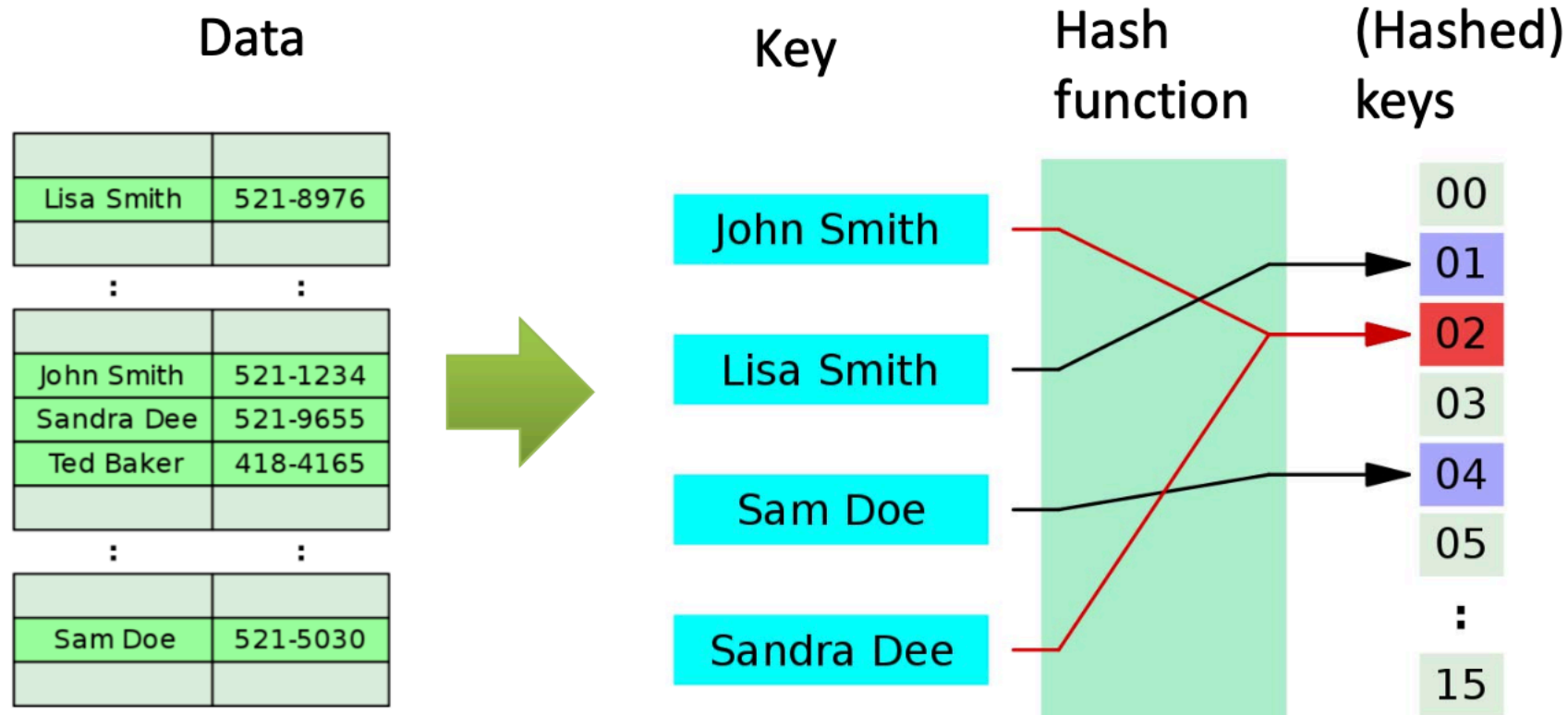
unit action: 369, 370, 371, 432, 453, 460,

461, 467, 469, 537, 538, 579, 580, 541



Hash

Hash function maps (partial) data into (hashed) keys for following-up indexing



HBS: A naive hash function

Let's assume: A = 1, C = 2, G = 4, T = 8, then: $HBS(S) = \sum_i HBS(S_i)$, e.g:

$HBS(AAAAA) = 1 + 1 + 1 + 1 + 1 = 5$

$HBS(GTACG) = 4 + 8 + 1 + 2 + 4 = 19$

...

123456789012345678901234567890
TAACCCTAACCCTAA...AACCCTAACCC

Reference Genome

Index
Table

...	
20	
...	

2+2+8+4+4
=20

Address
Table
(CCTAA, 11)
...

CCTAA

HBS

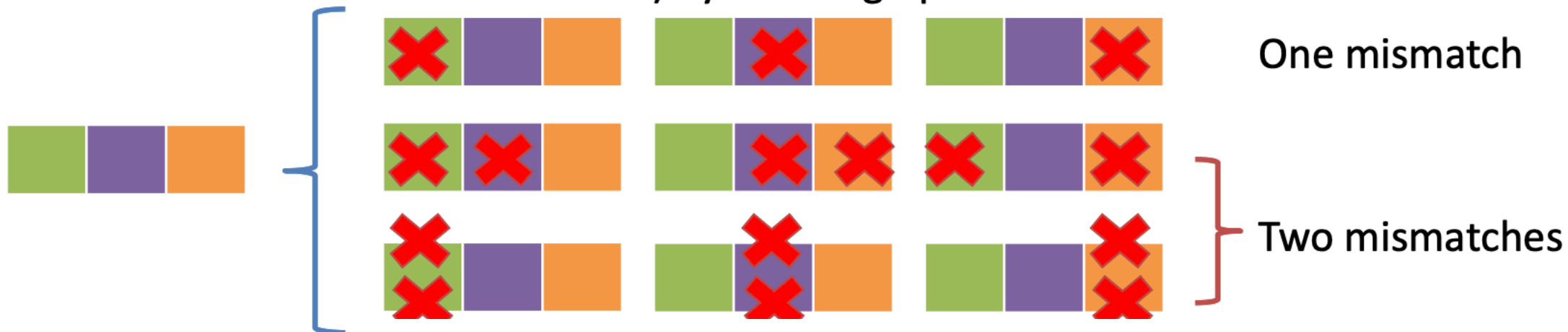
Pigeonhole principle (抽屉原理)

“In mathematics, the pigeonhole principle states that if n items are put into m pigeonholes with $n > m$, then at least one pigeonhole must contain more than one item.”



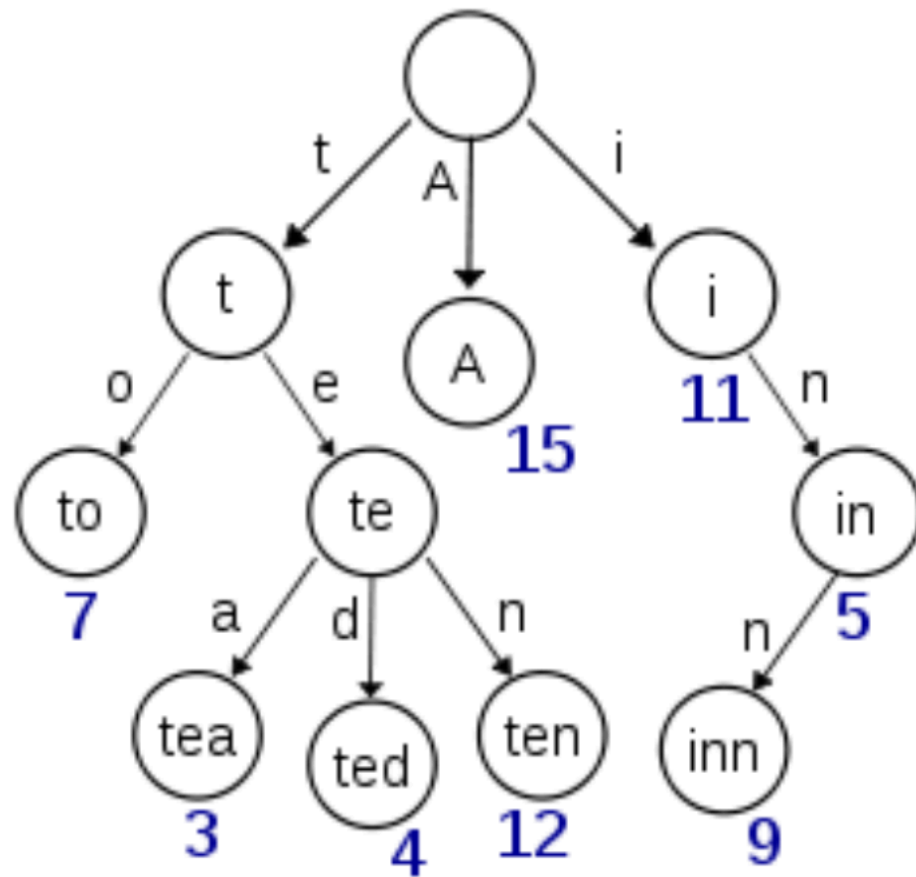
http://en.wikipedia.org/wiki/Pigeonhole_principle

After splitting the read into n (non-overlapped) blocks, there will be **at least $n-m$ perfectly-matched blocks** (i.e. without any mismatch with in the block) by allowing up-to- m mismatches.



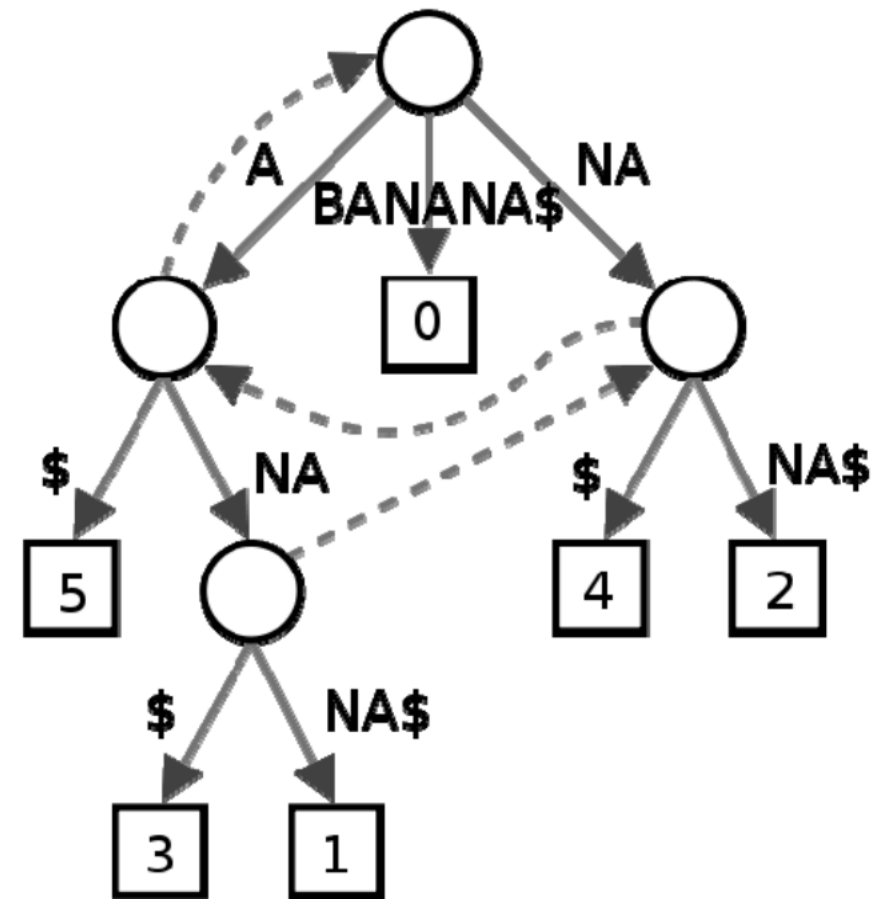
ELAND
MAQ
SOAP1
...

Prefix Tree



<http://en.wikipedia.org/wiki/Trie>

Suffix Tree



http://en.wikipedia.org/wiki/Suffix_tree

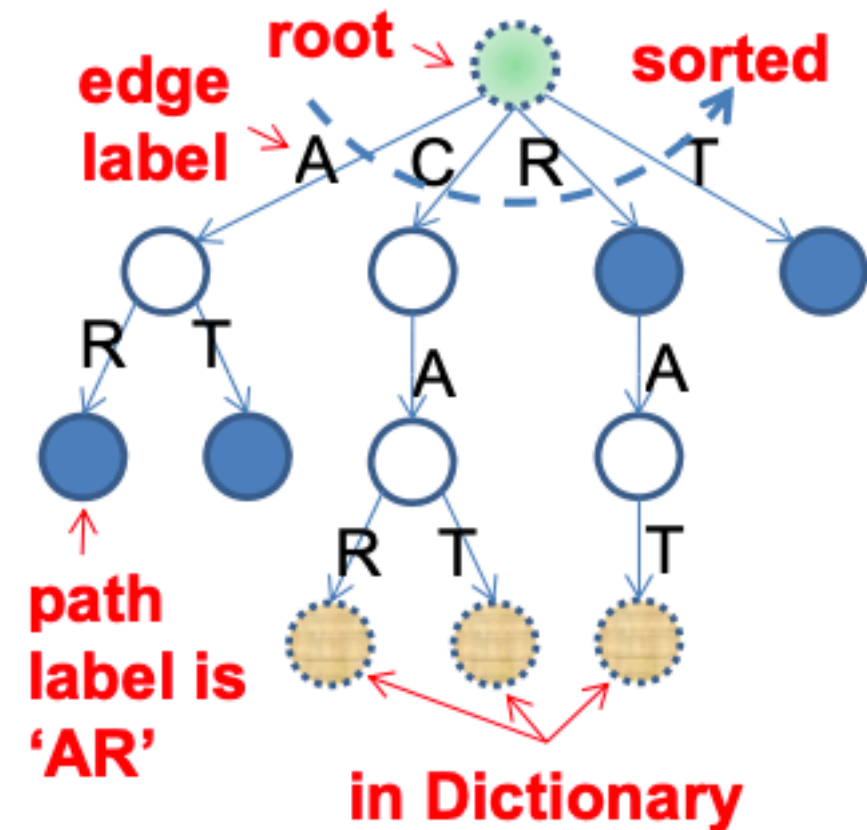
Suffix Trie ('CAR', 'CAT', 'RAT')

All Suffixes:

1. CAR
2. AR
3. R
4. CAT
5. AT
6. T
7. RAT
8. AT
9. T

Sorted Unique Suffixes:

1. AR
2. AT
3. CAR
4. CAT
5. R
6. RAT
7. T



Suffix Array (1)

- Suffix Array (SA) is an array that stores:
 - A permutation of n indices of sorted suffixes
 - Each integer takes $O(\log n)$ bits, so SA takes $O(n \log n)$ bits
- e.g. consider $T = \text{'GATAGACA\$'}$

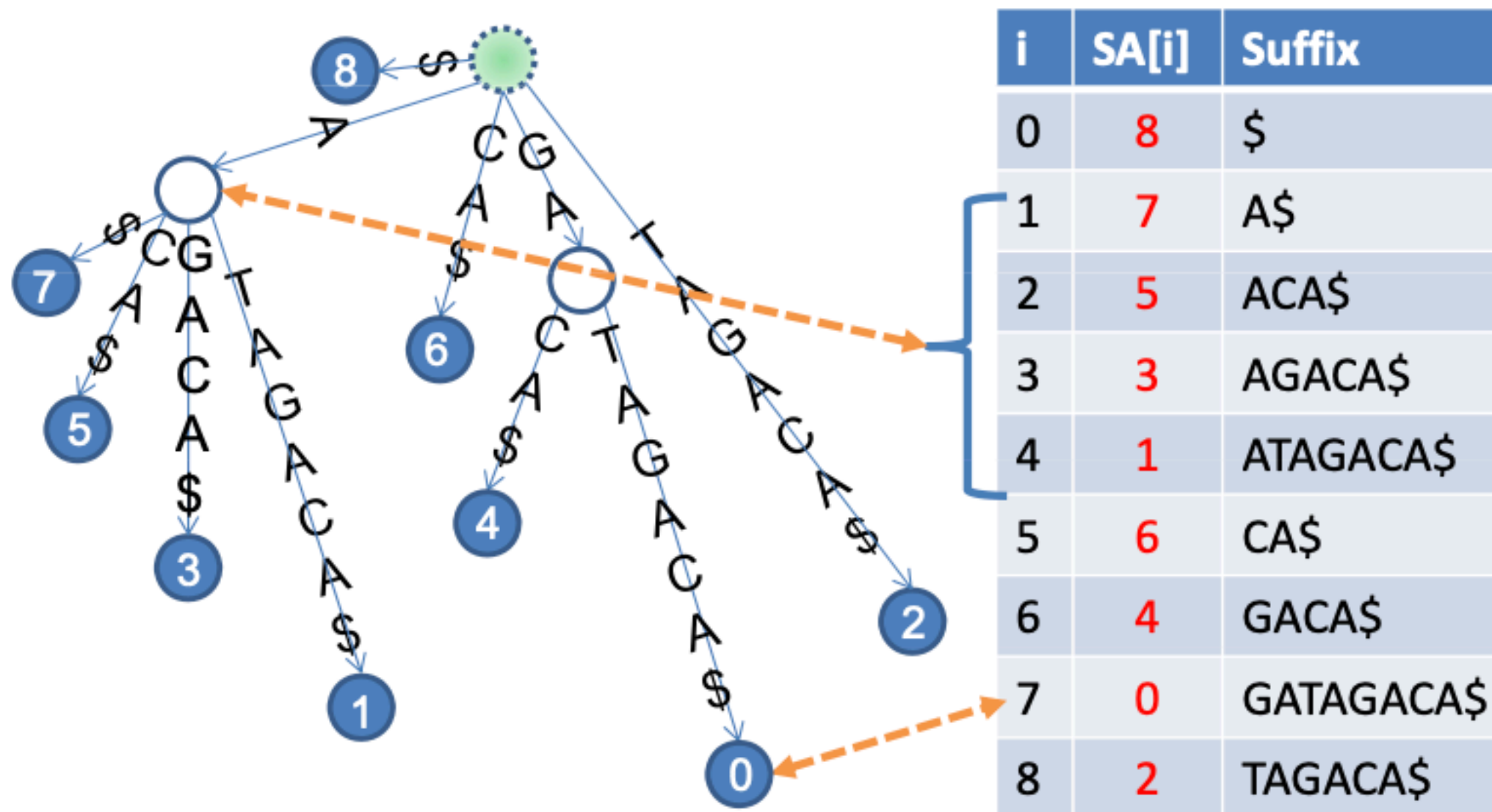
i	Suffix
0	GATAGACA\$
1	ATAGACA\$
2	TAGACA\$
3	AGACA\$
4	GACA\$
5	ACA\$
6	CA\$
7	A\$
8	\$

Sort →

i	SA[i]	Suffix
0	8	\$
1	7	A\$
2	5	ACA\$
3	3	AGACA\$
4	1	ATAGACA\$
5	6	CA\$
6	4	GACA\$
7	0	GATAGACA\$
8	2	TAGACA\$

Suffix Array (2)

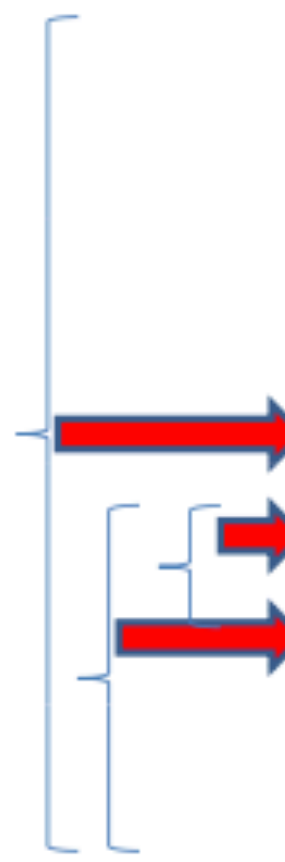
- Preorder traversal of the Suffix Tree visits the terminating vertices in Suffix Array order
- **Internal vertex** in ST is a **range** in SA
 - Each **terminating vertex** in ST is an **individual index** in SA = a suffix



String Matching Animation

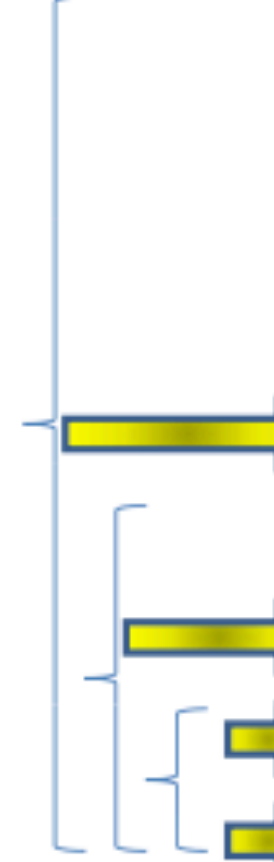
Finding $P = \text{'GA'}$

Finding lower bound



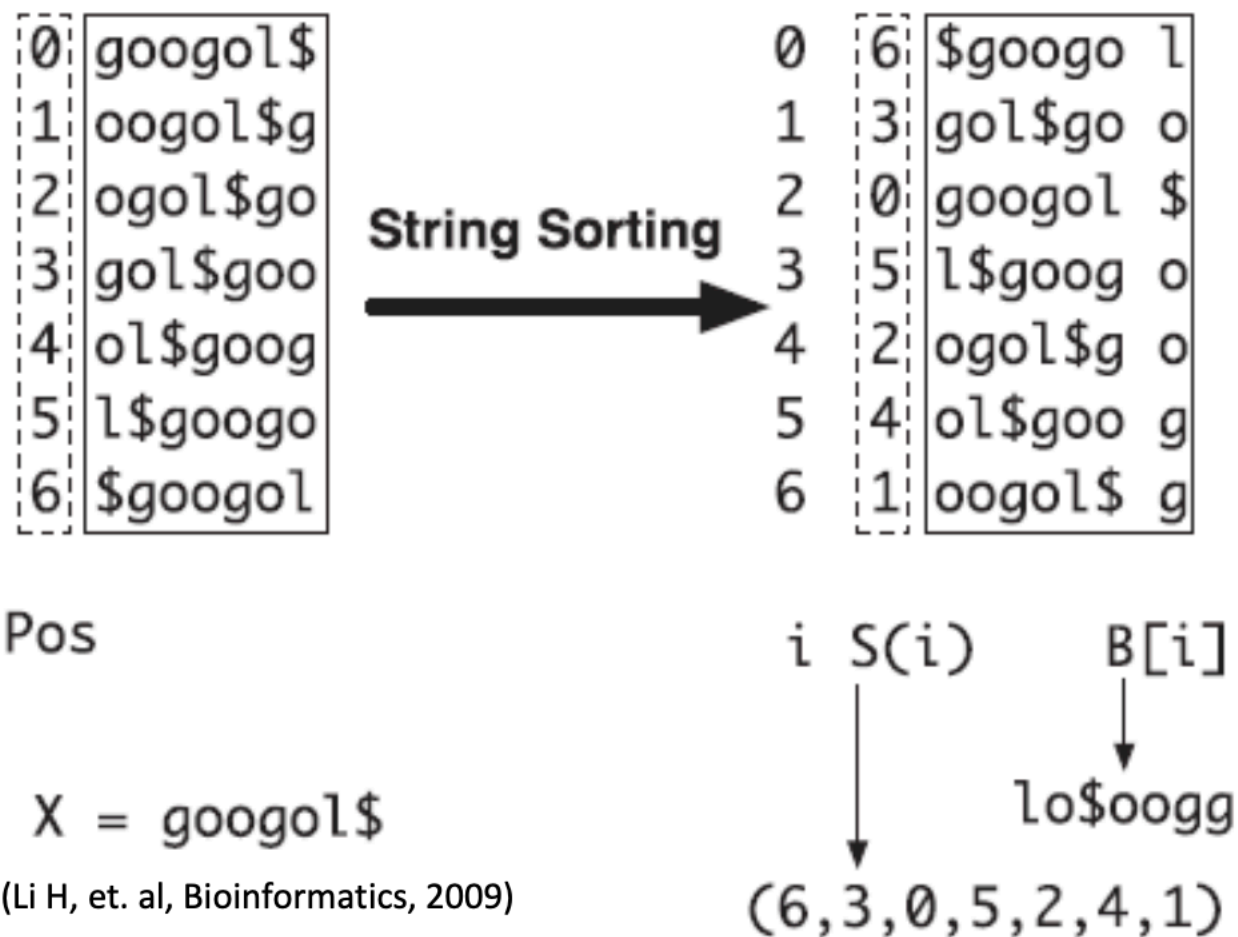
i	SA[i]	Suffix
0	8	\$
1	7	A\$
2	5	ACA\$
3	3	AGACA\$
4	1	ATAGACA\$
5	6	CA\$
6	4	GACA\$
7	0	GATAGACA\$
8	2	TAGACA\$

Finding upper bound



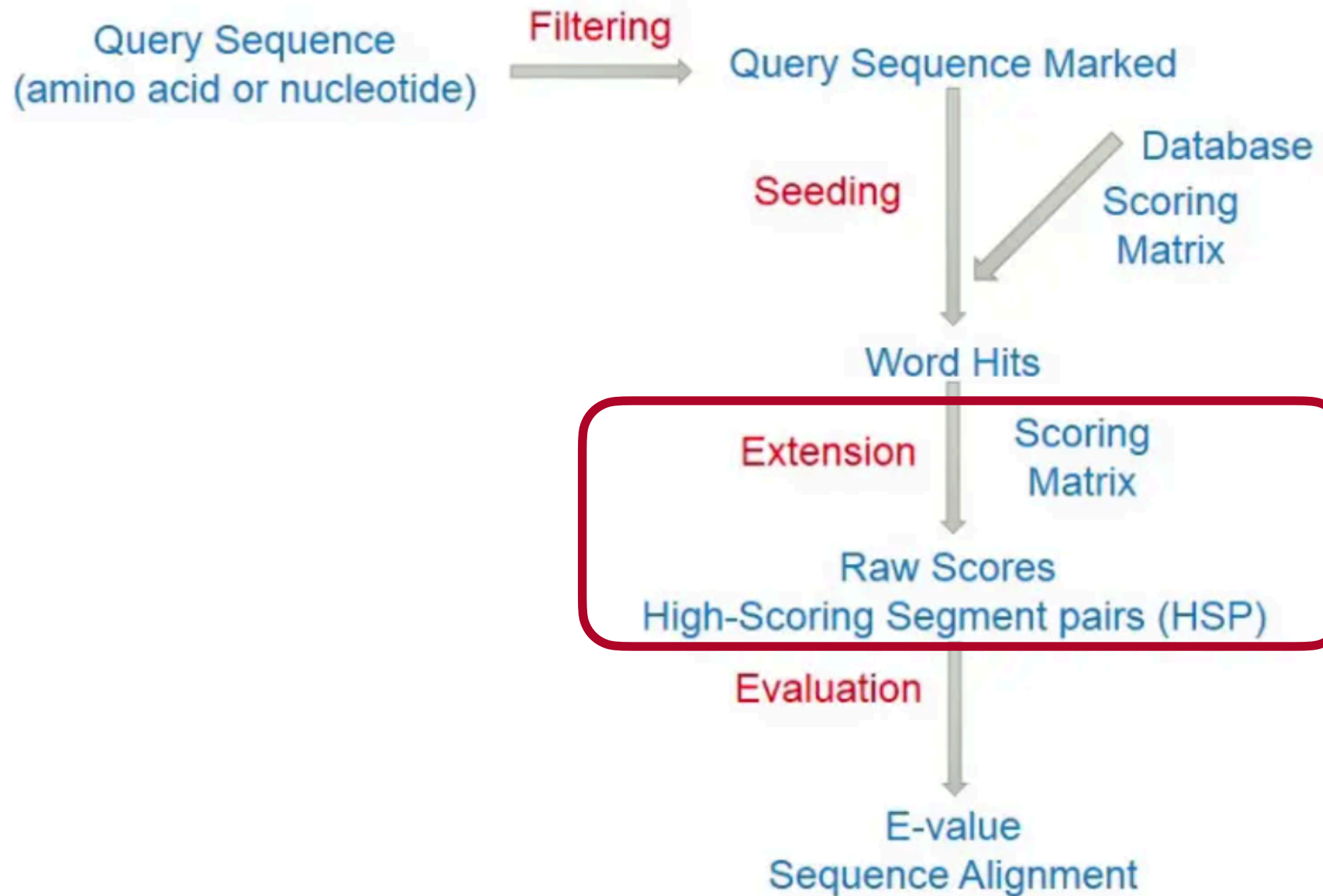
i	SA[i]	Suffix
0	8	\$
1	7	A\$
2	5	ACA\$
3	3	AGACA\$
4	1	ATAGACA\$
5	6	CA\$
6	4	GACA\$
7	0	GATAGACA\$
8	2	TAGACA\$

Burrows–Wheeler transform (BWT)



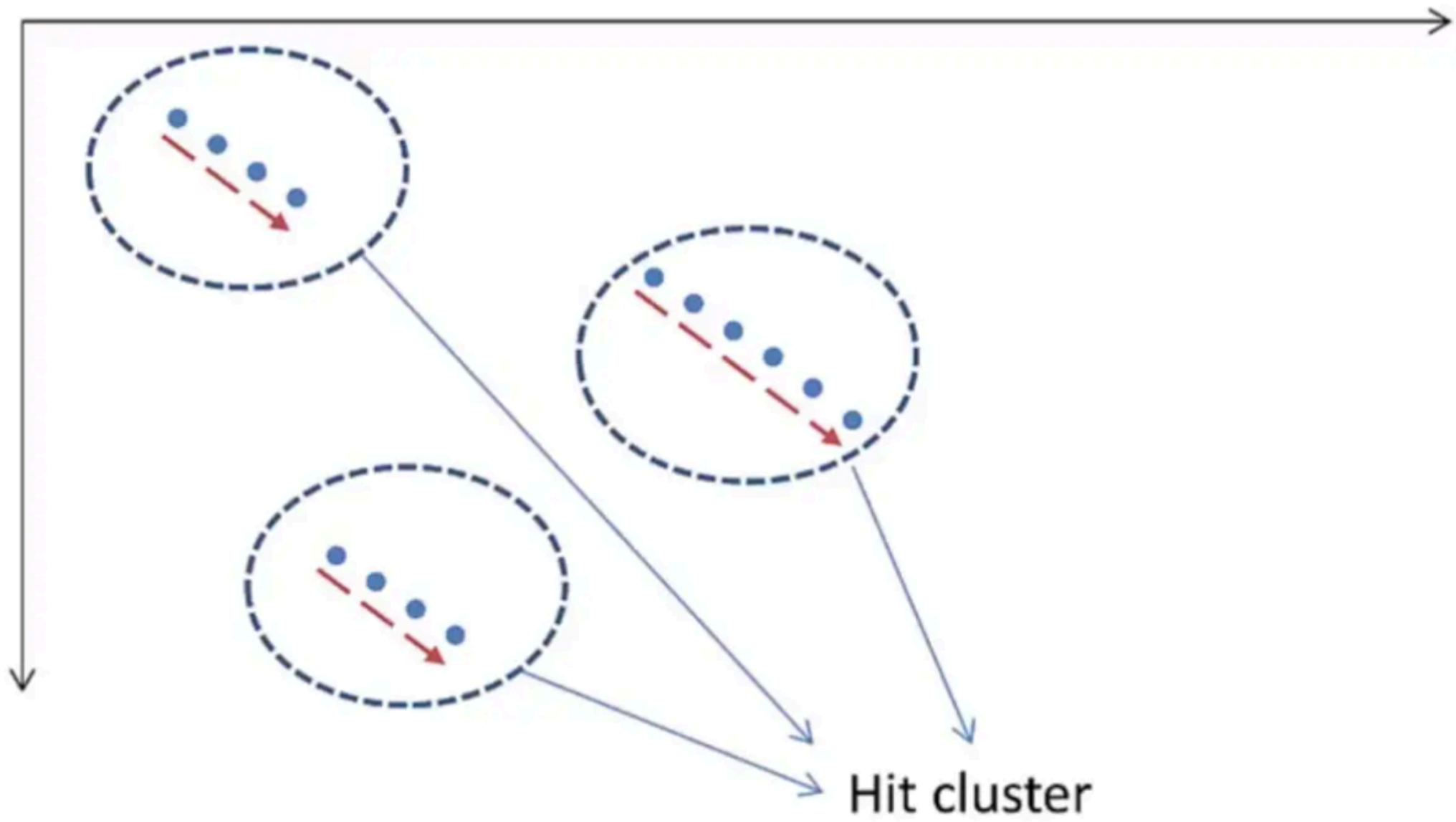
BOWTIE
BWA
SOAP3
...

BLAST

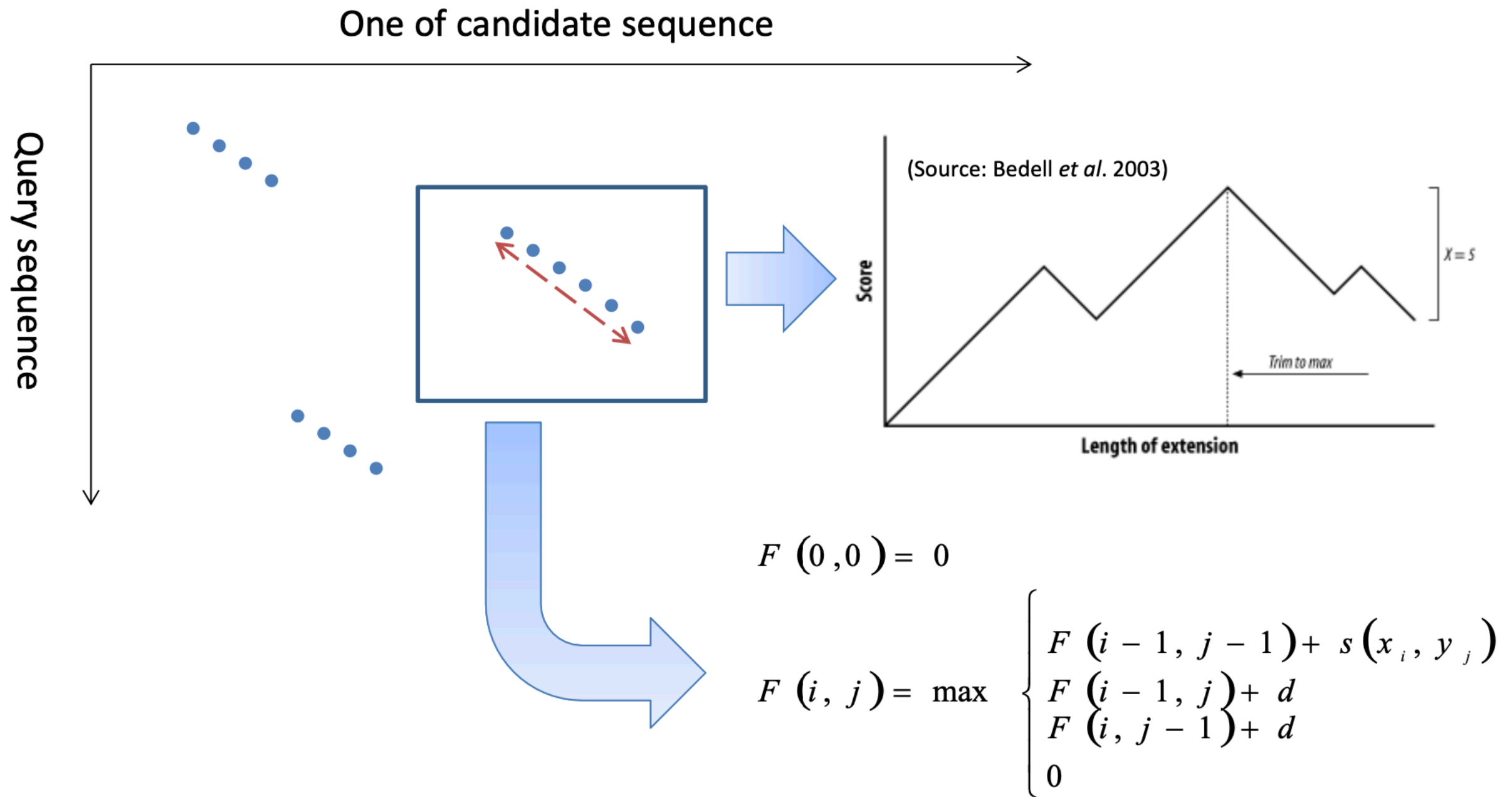


One of candidate sequence

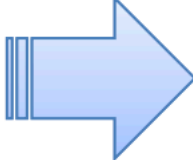
Query sequence



hit cluster



Quality: Given p = the probability of a base calling is *wrong*, its Quality Score can be written as

$$Q = -10 * \log_{10}(p)$$


p	Q
0.1	10
0.01	20
0.001	30
0.0001	40



0	10	20	30	40
!	"#\$%&'()	*+, - . / 0 1 2 3 4 5 6 7 8 9 :	; < = > ? @ A B C D E F G H I	
0	10	20	30	40

Mapping Quality

Given reference sequence z (length L), a read sequence x (length l), u is the alignment position of x on z , the probability that z actually coming from the position u is $p(z|x, u)$

(Genome Res. 2008 Nov;18(11):1851.)

$$p(z|x, u) = \prod_{\text{mismatch}} p(z_i) \quad SQ(u) = \log(p(z|x, u)) = \sum_{\text{mismatch}} p(z_i) = \sum_{\text{mismatch}} Q(z_i)$$

Read: ACGT (Quality: 30 30 25 20)

Ref: ACGTACGGA

ACGT	0 +	0 +	0 +	0	SQ (0)
ACGT	30 +	30 +	25 +	20	SQ (1)
ACGT	30 +	30 +	25 +	20	SQ (2)
ACGT	30 +	30 +	25 +	20	SQ (3)
ACGT	0 +	0 +	0 +	20	SQ (4)
ACGT	30 +	30 +	0 +	20	SQ (5)

Mapping Quality

If we assume that a **uniform NULL model**, i.e. the read can randomly come from all possible positions with equal probability, then the error of mapping to a specified position u could be written as

$$E(u) = \frac{SQ(u)}{\sum_i SQ(i)} \quad (\text{Genome Res. 2008 Nov;18(11):1851.})$$

Read: ACGT (Quality: 30 30 25 20)

Ref: ACGTACGGA

	<u>SQ (u)</u>	<u>E (u)</u>
ACGT	0 + 0 + 0 + 0	0 / 415
ACGT	30 + 30 + 25 + 20	105 / 415
ACGT	30 + 30 + 25 + 20	105 / 415
ACGT	30 + 30 + 25 + 20	105 / 415
ACGT	0 + 0 + 0 + 20	20 / 415
ACGT	30 + 30 + 0 + 20	80 / 415

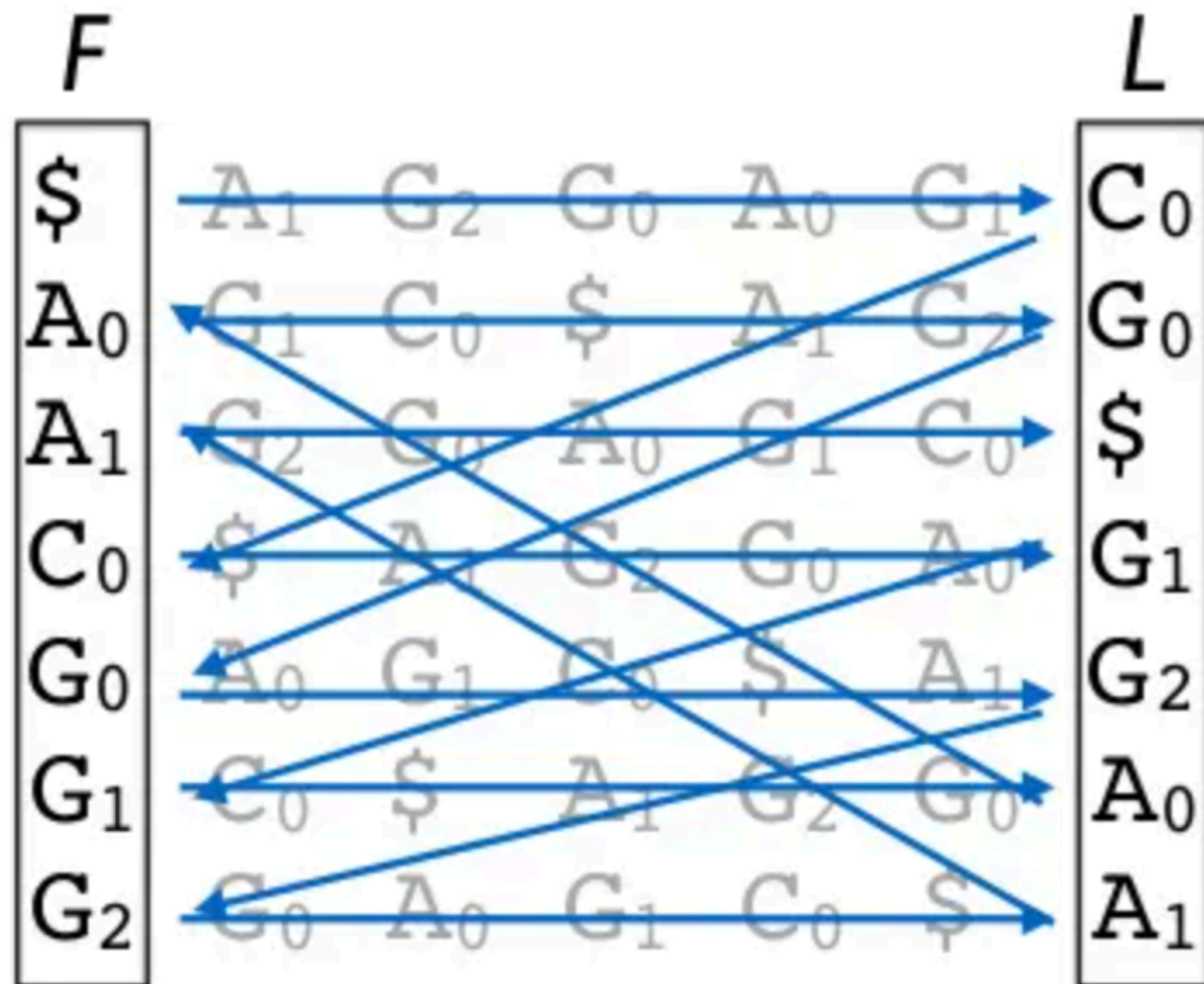
*More on BWA/BWT algorithm

- The compression algorithm used in BWA
- Lossless compression
- Sort and transform the char matrix with string rotation
- Reverse-char method was utilized for match
- Cannot handle gap

BWA/BWT Example: AGGAGC

No.	rotated	index	sorted	F	L	LF
0	AGGAGC\$	6	\$AGGAGC	\$	C	C->\$
1	GGAGC\$A	3	AGC\$AGG	A	G	G->A
2	GAGC\$AG	0	AGGAGC\$	A	\$	\$->A
3	AGC\$AGG	5	C\$AGGAG	C	G	G->C
4	GC\$AGGA	2	GAGC\$AG	G	G	G->G
5	C\$AGGAG	4	GC\$AGGA	G	A	A->G
6	\$AGGAGC	1	GGAGC\$A	G	A	A->G

1. According to index, $L \rightarrow F$
2. Infer ref sequence according to L and F



To implement BLAST in R

```
# 导入Biostrings包
library(Biostrings)
# 定义查询序列和数据库序列
query_seq <- DNASTring("ATCGATCGATCGATCG")
db_seq <- DNASTring("CGATCGATCGATCGATC")
# 定义k-mer的长度
k <- 3
# 将查询序列和数据库序列分别转换为数字编码
query_seq_num <- as.numeric(query_seq)
db_seq_num <- as.numeric(db_seq)
# 将查询序列和数据库序列分别划分成k-mer
query_kmer <- kmer(query_seq_num, k)
db_kmer <- kmer(db_seq_num, k)
# 对于每个查询序列的k-mer, 查找数据库序列中所有与之匹配的k-mer
matches <- matchPattern(query_kmer, db_kmer)
# 对于每个匹配的k-mer, 计算查询序列和数据库序列之间的相似度得分
scores <- pmatch(query_kmer, db_kmer, fixed=FALSE)
# 对于每个查询序列的k-mer, 选择相似度得分最高的匹配序列, 并将其作为最佳匹配
best_matches <- maxMatches(matches)
# 对于每个最佳匹配, 计算匹配序列的长度、相似度得分、E值等参数
match_length <- width(best_matches)
match_score <- scores[best_matches]
e_value <- length(db_kmer) * (1 - exp(-match_score))
# 根据E值和相似度得分, 对匹配结果进行排序, 输出最终的比对结果
result <- data.frame(query_seq, db_seq, match_length, match_score, e_value)
result <- result[order(result$e_value),]
```

To implement BLAST in Python

```
pip install biopython
```

```
from Bio.Blast import NCBIWWW
from Bio.Blast import NCBIXML
# 进行BLAST比对
result_handle = NCBIWWW.qblast("blastn", "nt", "ACGTGAGGCTAGCTAGCTAGCTAGCTAGCTAGCTAGCTAGCTAGCTAGCTAGCTAGCTAGC")
# 读取BLAST比对结果
blast_record = NCBIXML.read(result_handle)
# 输出比对结果
for alignment in blast_record.alignments:
    for hsp in alignment.hsps:
        print('****Alignment****')
        print('sequence:', alignment.title)
        print('length:', alignment.length)
        print('e value:', hsp.expect)
        print(hsp.query[0:75] + '...')
        print(hsp.match[0:75] + '...')
        print(hsp.sbjct[0:75] + '...')
```



THANK YOU